Unit 6
Timing and Critical Path Analysis

ECE-GY 6463: ADVANCED HARDWARE DESIGN

PROFS. SUNDEEP RANGAN, SIDDHARTH GARG

Learning Objectives

Skill	You will be able to
Reason about signal stability	Find when a signal is unknown after its input changes, and its stable values before and after
Compute propagation delay	Find when each signal settles: delays in series add, parallel paths wait for the latest, and a mux waits for every input
Perform static timing analysis	Find the critical path between registers and the maximum clock frequency, with clock-to-Q and setup time
Read and draw timing diagrams	Draw detailed and cycle-accurate diagrams, and describe a waveform by its value-change points
Decompose a function across clock cycles	Split a datapath into registered stages, and trade latency for clock rate

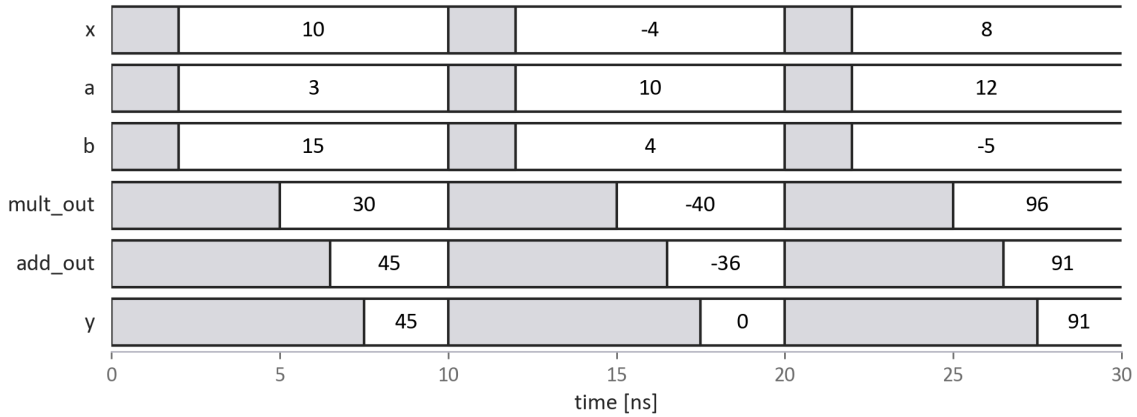
Each skill is assessed by problems in the LLM grader. The demos add what no problem tests yet: extracting a VCD file from a Vitis simulation and plotting it in Python.

Outline

- Static Timing Analysis in Combinational Logic
- Static Timing Analysis in Sequential Logic
- Sequential Logic Timing Diagrams
- VCD Files and Timing Diagrams

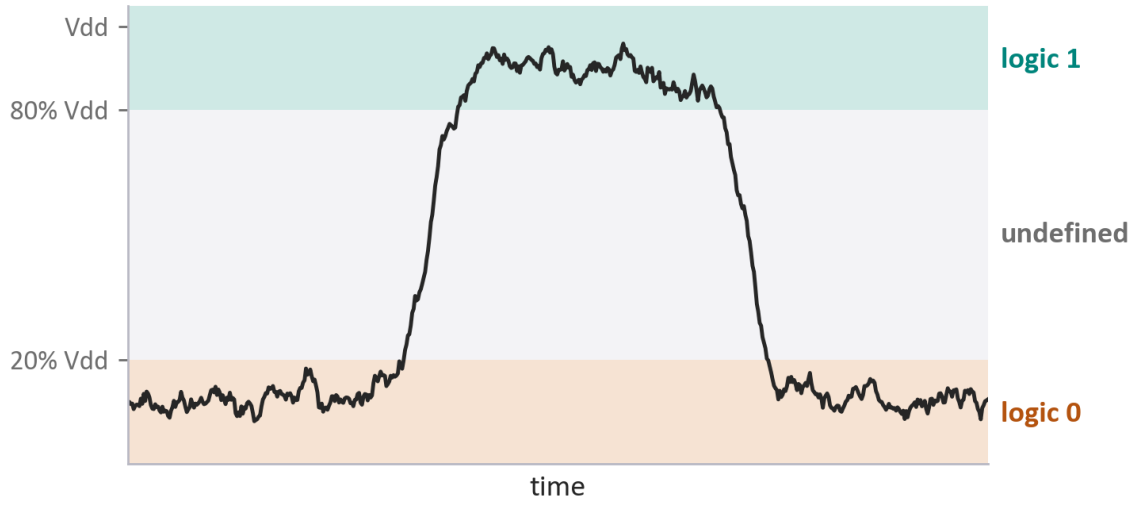
Static Timing Analysis

- **Timing analysis**: how long signals take to pass through a circuit
  - Drawn as a **timing diagram**
- It sets the clock speed, the latency and the throughput
- **Static** timing analysis (STA)
  - Uses each block's **maximum** propagation delay
  - Holds for every input: not data dependent
- Conservative, but a hard guarantee: the default in industry



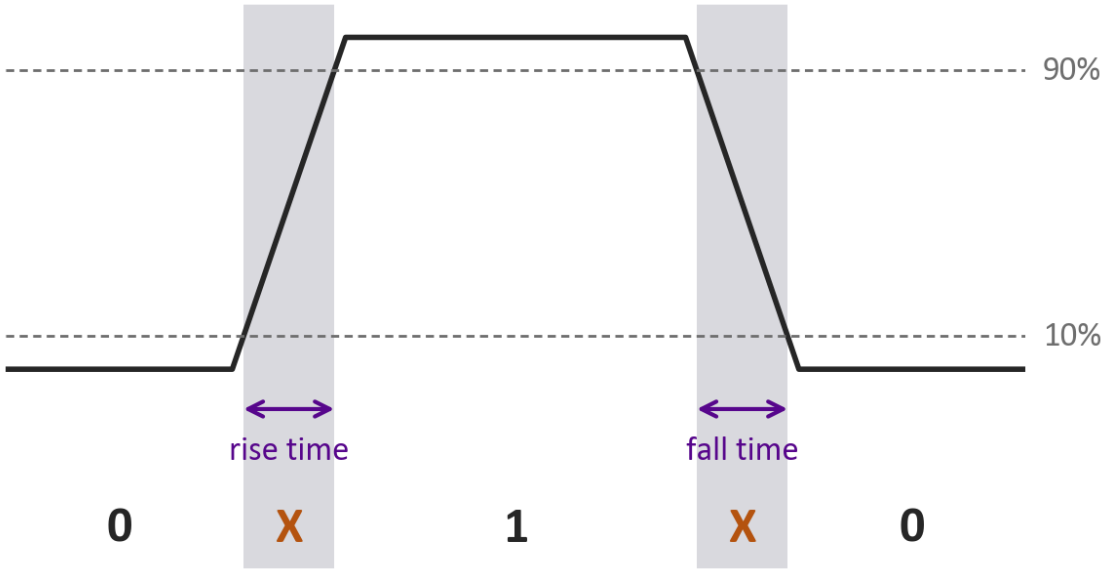
Implementing a Signal Physically

- To understand timing, look at how a signal is carried physically
- A **binary signal** has two logical values: 0 and 1
- Physically, it is usually a **voltage** on a wire
  - The voltage takes continuous values
  - The logical value is read by **thresholding** it

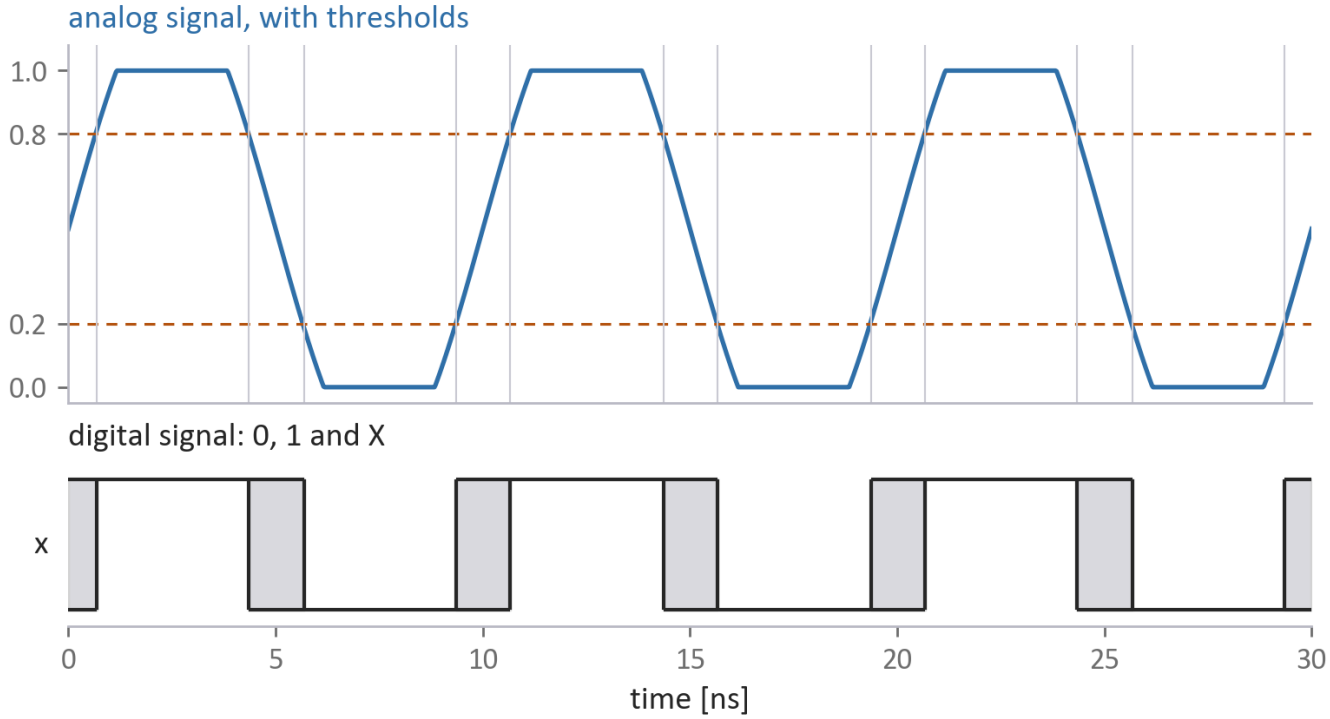


Undefined Logic Value: X or ?

- Signals take time to change: they are outputs of physical circuits
- In between, the value is **undefined**
  - Also called unknown or indeterminate
- Written **X**, or **?** to avoid confusion with a variable named x
- So a logic signal has three values: 0, 1 and X
  - A fourth, Z (high impedance), comes later



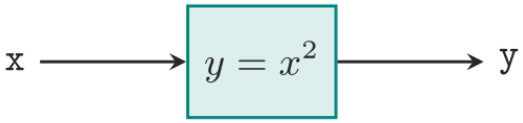
Undefined Regions Illustrated



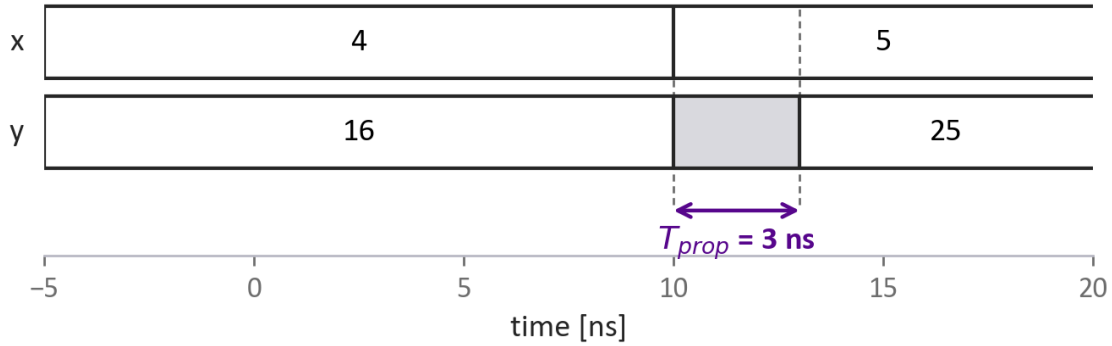
- **Analog** signal: a continuous voltage, with finite transitions
- **Digital** signal: 0 below 20%, 1 above 80%, X in between
- X is drawn **grey** in every timing diagram in this unit

Stable Values and Propagation Delay

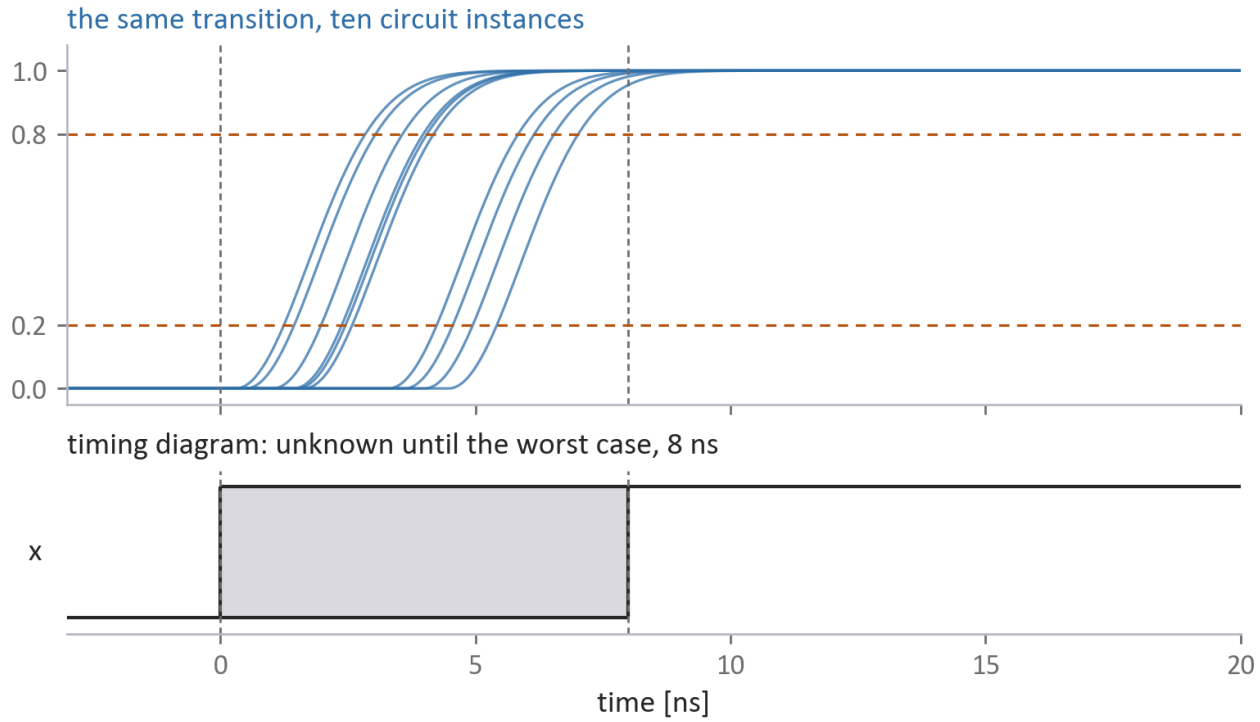
- **Stable value**: the value once a transition settles
  - $y = x^2$, with x changing from 4 to 5: y settles at 25
- **Propagation delay**: the time the output takes to settle
- It has two parts
  - **Logic delay**: the gates settling
  - **Routing delay**: the signal crossing the wires
  - Both depend on capacitance and drive current



$T_{\text{prop}} = 3$ **ns**



Maximum Propagation Delay



- Actual delays vary: wire length, temperature, the device
- Timing diagrams use the **maximum** propagation delay
- X marks where the value is **not guaranteed**
  - Often stable well before the maximum
- The maximum is conservative
  - Real delays may be far below it
  - Exceeding it is usually catastrophic

Propagation Delay

PRACTICE

A signal x is unknown before 10 ns. It is 0 from 10 ns, 4 from 20 ns and 2 from 30 ns.

It passes through a circuit whose stable output is y = 2x + 1, with a propagation delay of 3 ns.

List the times at which y changes and the value it takes at each. Mark the periods where y is unknown, y = ?.

The input

Time [ns]	10	20	30
x	0	4	2

Also in the LLM grader: Unit 6 › Propagation delay

Propagation Delay

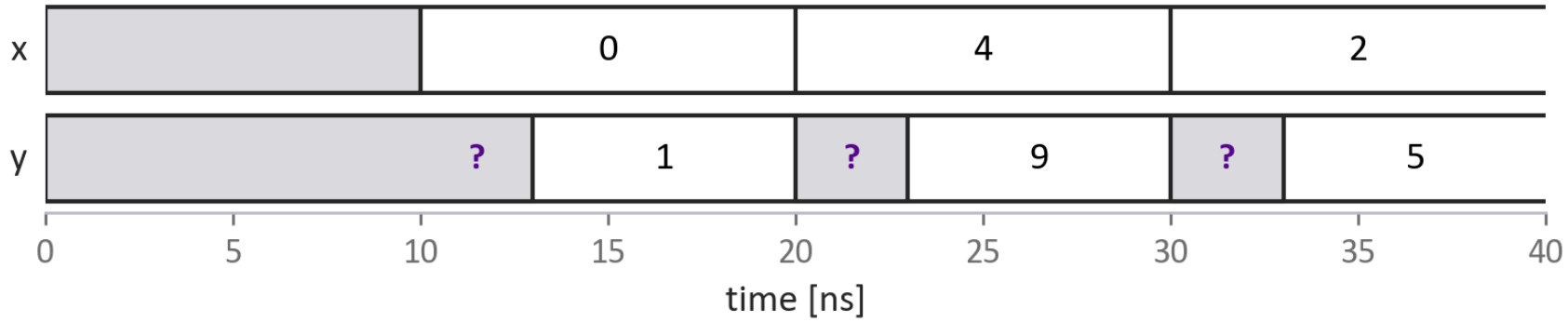
SOLUTION

After each change of x, y is unknown for 3 ns, then settles to 2x + 1: **1, 9 and 5**

Time [ns]	10	13	20	23	30	33
y	?	1	?	9	?	5

Common error

Listing only (13, 1), (23, 9), (33, 5): the unknown periods are part of the answer.

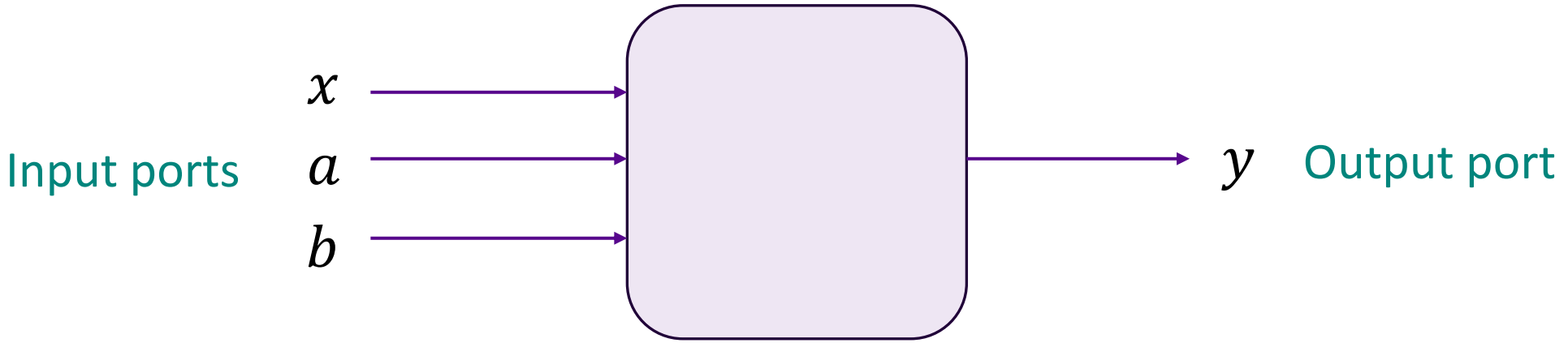


Also in the LLM grader: Unit 6 › Propagation delay

Timing in a Series Circuit: ReLU + Linear

As an example, consider a more complicated circuit: $y = \max\{ax + b, 0\}$

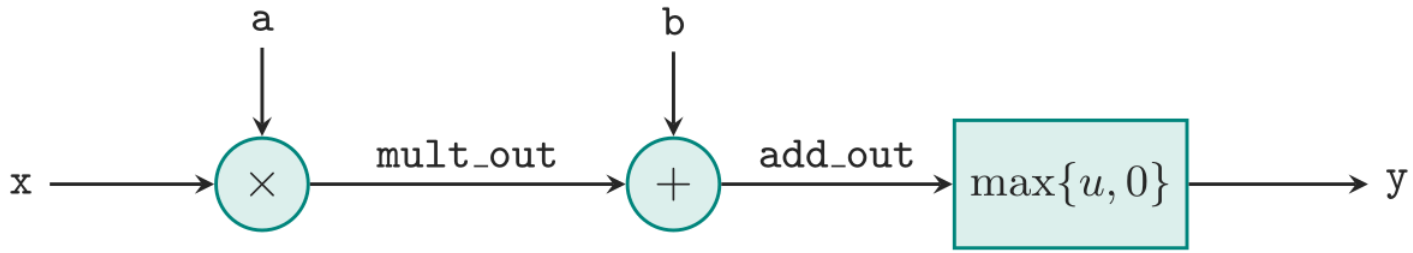
- Linear function + ReLU from Unit 2
- Used widely in machine learning



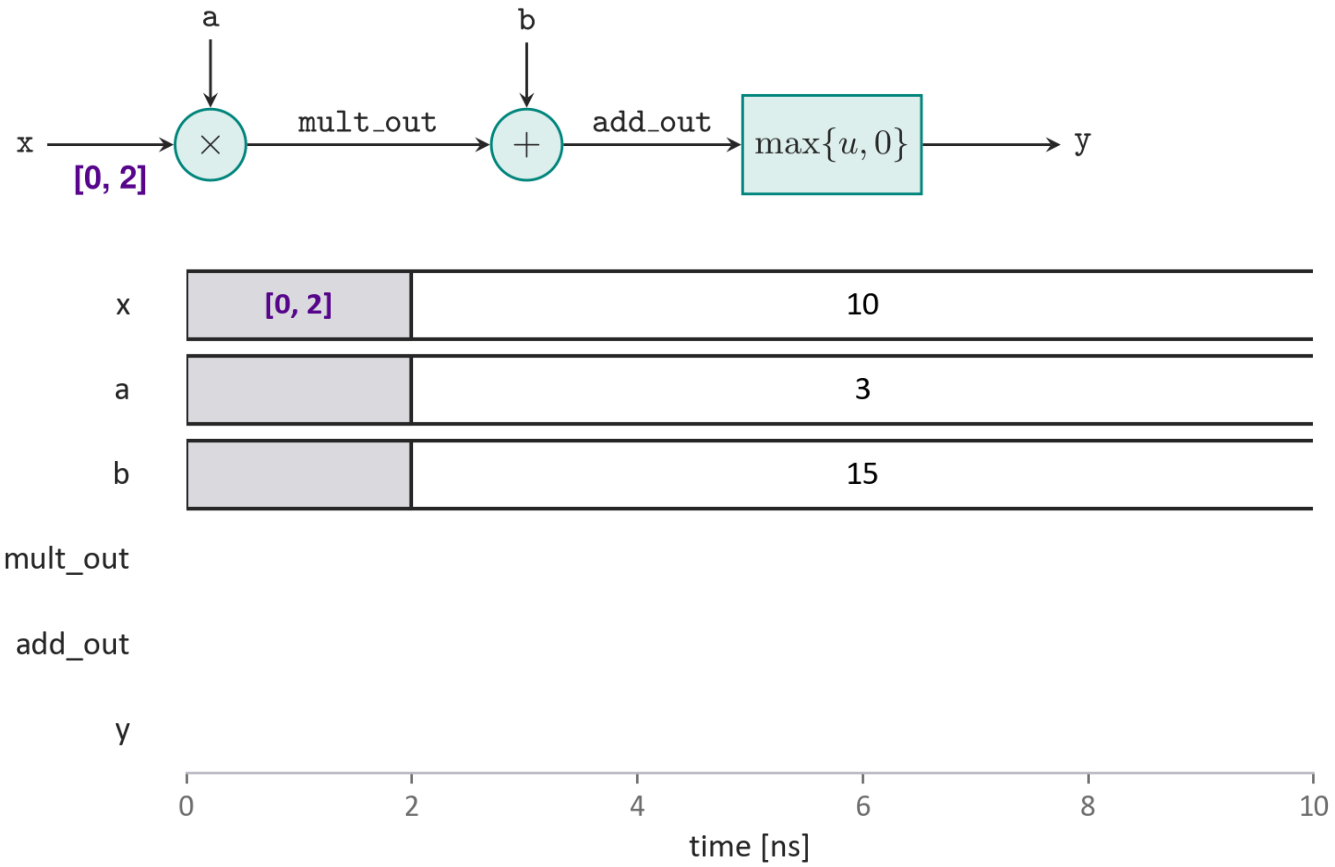
Timing Diagram for Series Connections

- Linear + ReLU is a **series** connection: three operations, one after another
- Assume the maximum propagation delays in the table
- What does the timing diagram look like when the inputs change?

Hardware block	Max prop delay
Multiplier	T_{mult} = 3 ns
Adder	T_{add} = 1.5 ns
Max operator	T_{max} = 1 ns
Total	**T_{tot} = 5.5 ns**

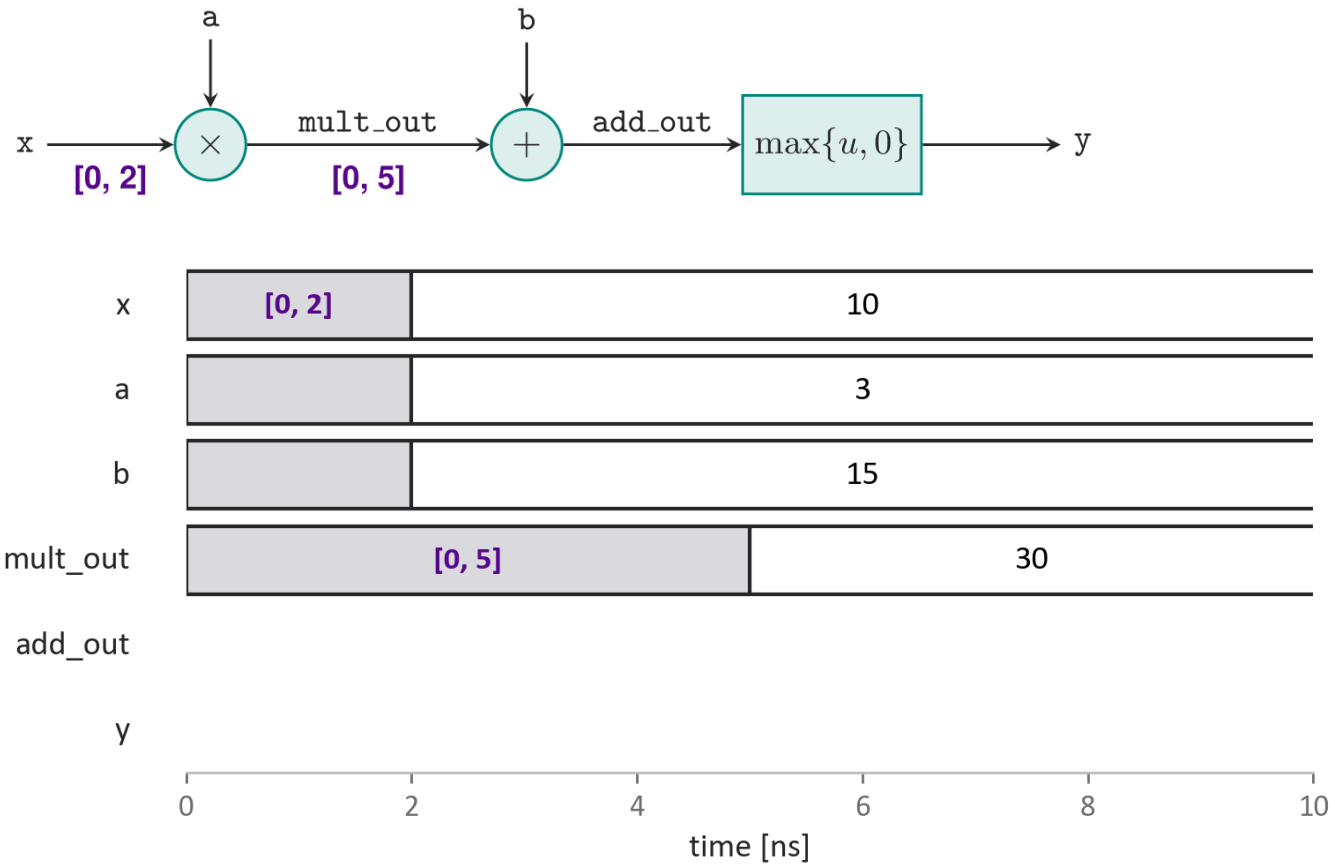


Timing Diagram of a Series



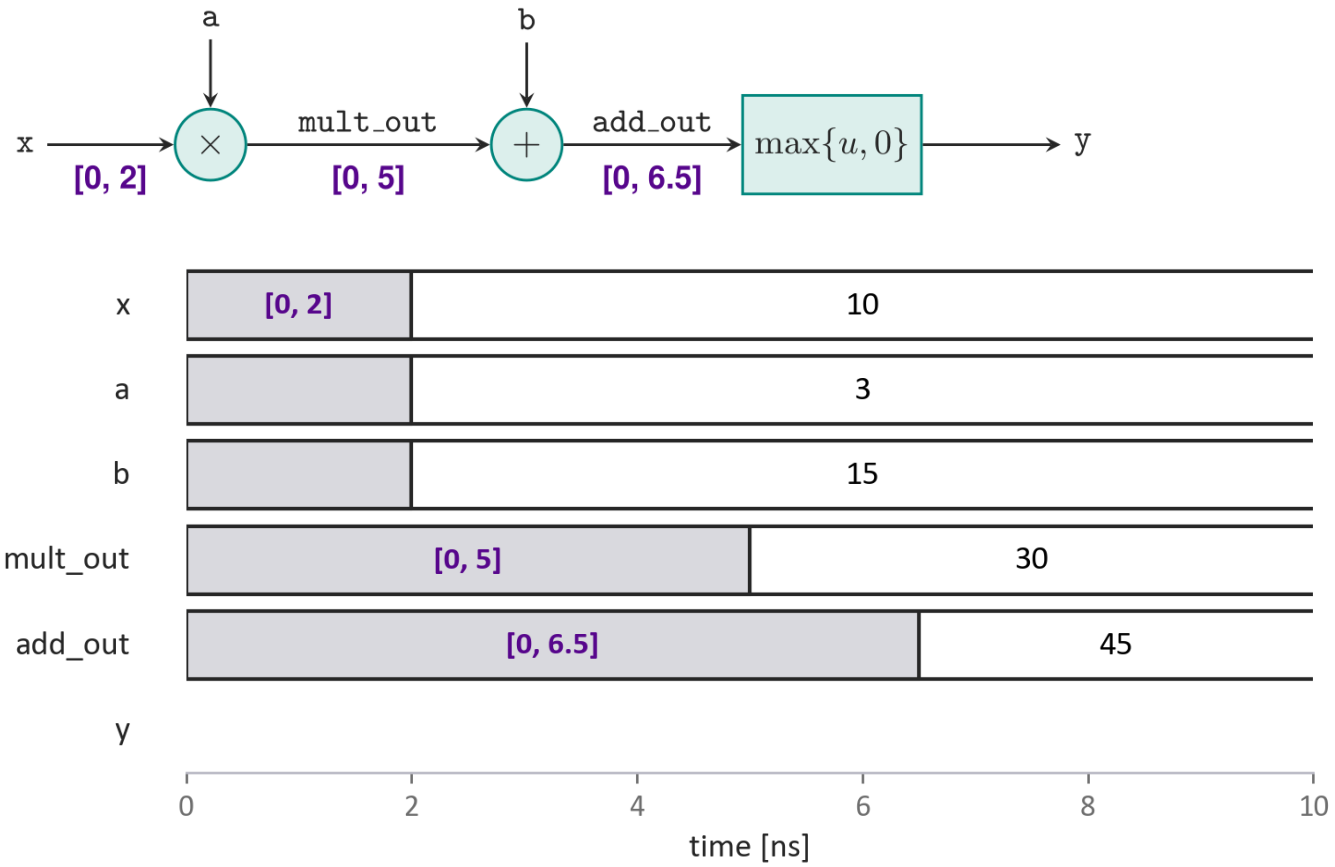
- Suppose x, a and b are in transition for t ∈ [0, 2]
  - They are unknown, X, there
- The unknown interval is written in **violet**
- When are the downstream signals stable?

Series Timing: First Block



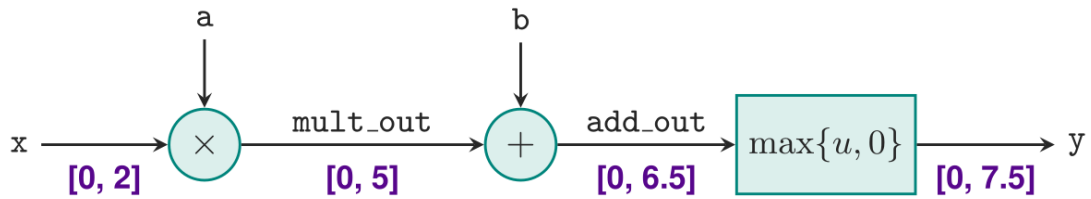
- Start with the multiplier
- Its inputs are unknown in [0, 2]
- Its maximum delay is 3 ns
- So mult_out is unknown in **[0, 5]**: 3 ns longer than its inputs
- Stable value: 3 × 10 = 30

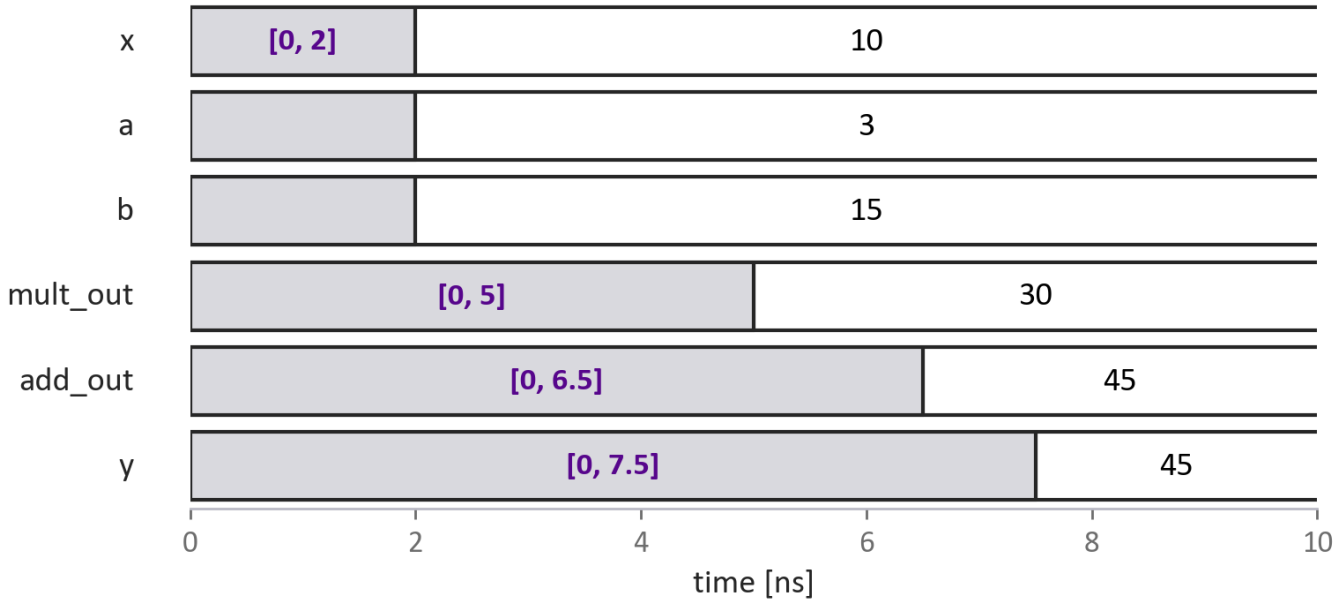
Series Timing: Second Block



- mult_out is unknown in [0, 5]
- The adder takes 1.5 ns, so add_out is unknown in **[0, 6.5]**
- add_out goes unknown **at t = 0**, as soon as x does
- **Unknown propagates at once; stability propagates with delay**

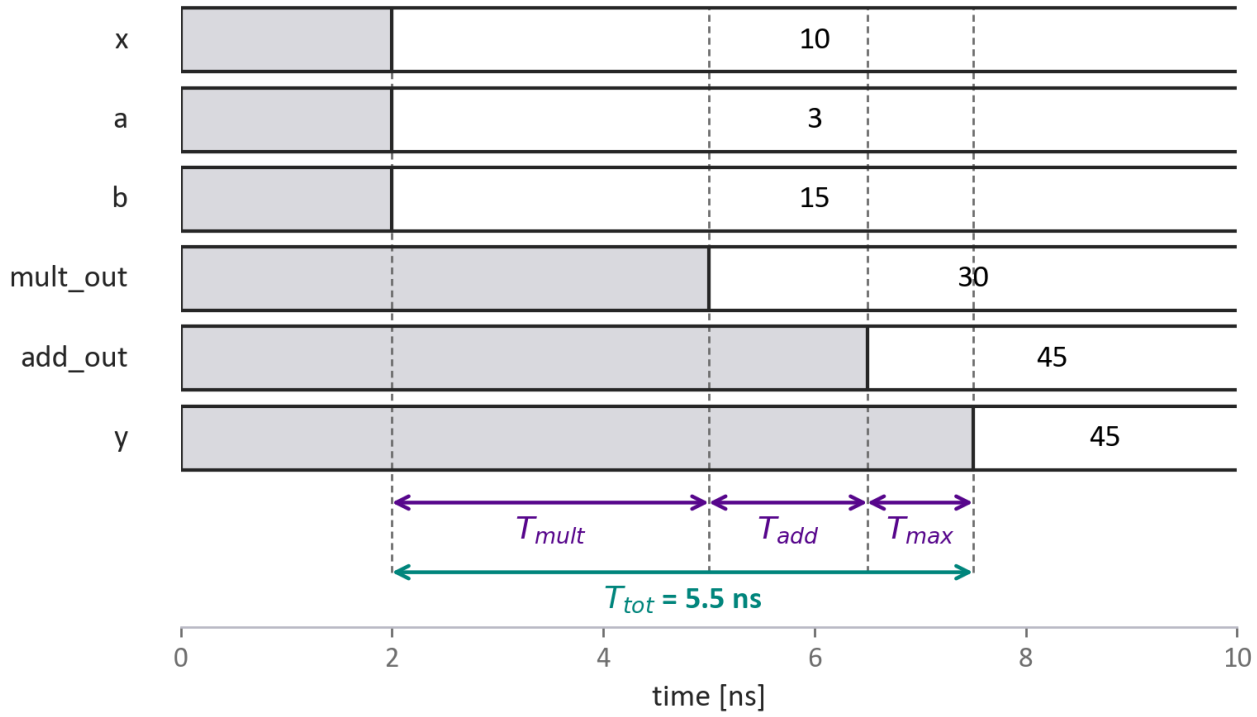
Series Timing: Third Block





- Like add_out: the max block takes 1 ns
- y is unknown in **[0, 7.5]**
- Total propagation delay: **5.5 ns**, counted from when the inputs are stable

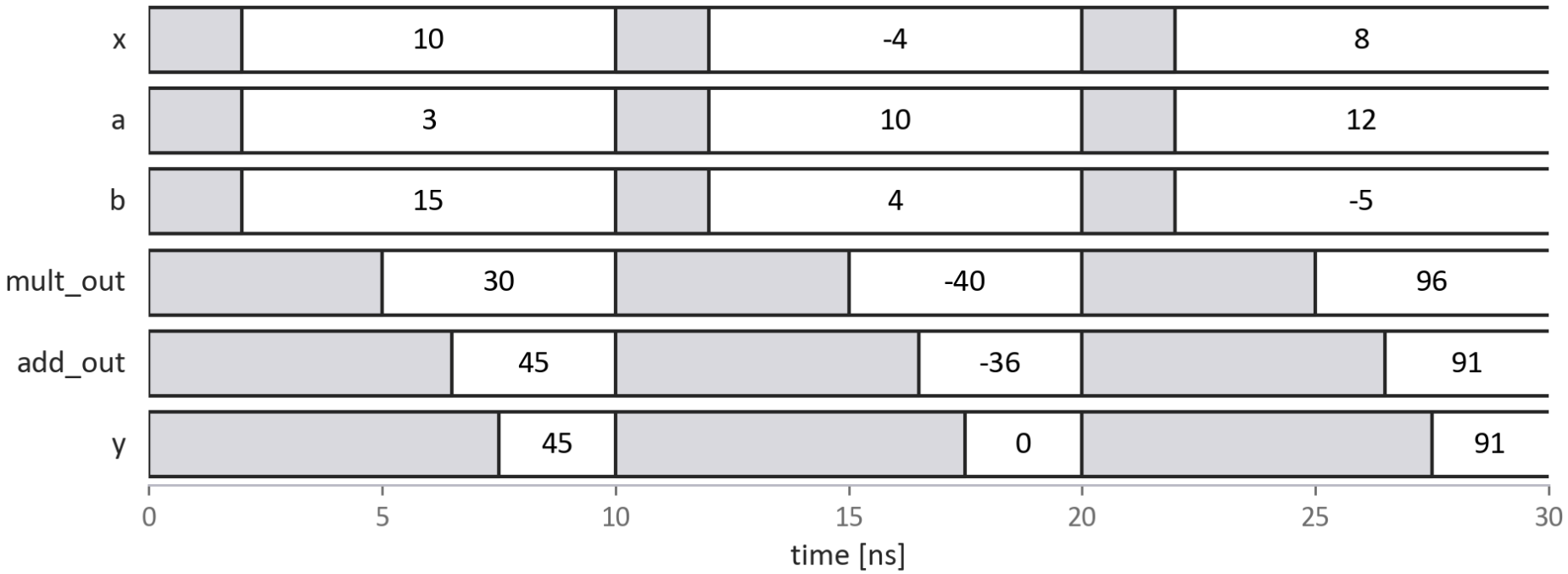
Example Total Propagation Delay



Hardware block	Example value
Multiplier	T_{mult} = 3 ns
Adder	T_{add} = 1.5 ns
Max operator	T_{max} = 1 ns
Total	**T_{tot} = 5.5 ns**

Along a series path the delays **add**: $T_{tot} = T_{mult} + T_{add} + T_{max}$

Example Timing Over Multiple Inputs



- Each change of the inputs repeats the pattern: y is unknown until 5.5 ns after the inputs settle
- Stable values: max(10·3 + 15, 0) = 45, max(−4·10 + 4, 0) = 0, max(8·12 − 5, 0) = 91

Series Timing

PRACTICE

Each multiplication has a maximum propagation delay of 3 ns and the addition 2 ns. x = 0 for a long time before 10 ns, then changes to x = 4 at 10 ns and stays there. a = 3 and b = 6 are constants.

(a) What are the stable values of xsq, y and z before and after the change?

(b) When does each of xsq, y and z become unknown (?), and when is it stable again at its new value?

SystemVerilog, combinational

```
xsq = x*x;
y = a*xsq;
z = y + b;
```

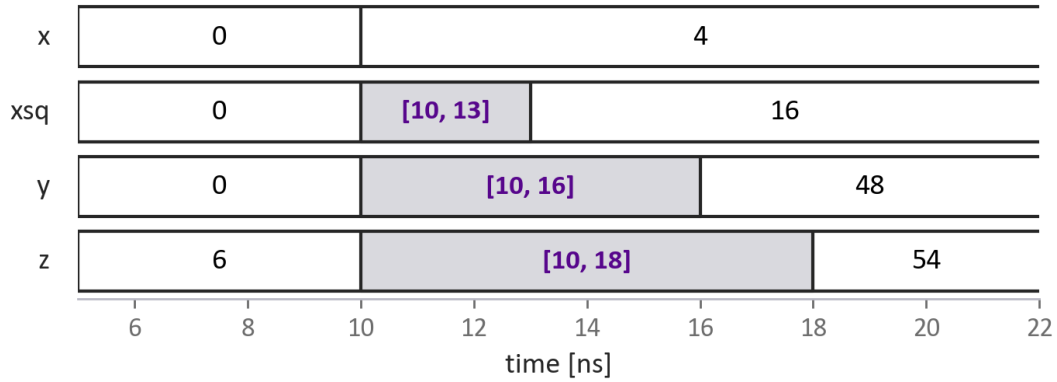
Also in the LLM grader: Unit 6 › Series timing

Series Timing

SOLUTION

(a) Before: xsq = 0, y = 0, z = 6
After: **xsq = 16, y = 3·16 = 48, z = 48 + 6 = 54**

(b) All three go unknown at **10 ns**, when x changes
xsq stable at **13 ns**: 10 + 3
y stable at **16 ns**: 13 + 3
z stable at **18 ns**: 16 + 2



Common error

y and z go unknown at 10 ns, not when the block before them settles.

Also in the LLM grader: Unit 6 › Series timing

Typical Propagation Delays: Low-End FPGA

Hardware block	FPGA resource	Typical range
32-bit adder	LUT + dedicated carry chain	1.5–3.0 ns
16-bit multiplier	DSP48E1 block	2.5–5.0 ns
16-bit comparator	LUT logic + carry chain	1.0–2.0 ns

- Rough delays for a low-end FPGA (e.g., Pynq Z2)
- We will explain the FPGA resources more in later units
- Routing delays not included
  - May be large for more complex designs

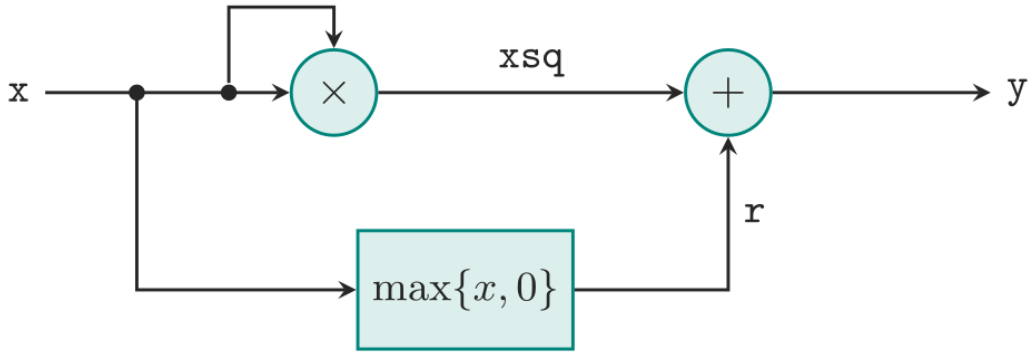
Typical Propagation Delays: ASIC

Hardware block	Typical structure	Typical range
32-bit adder	Kogge-Stone / Brent-Kung / hybrid	150-450 ps
16-bit multiplier	Wallace/Dadda tree + final adder	300-700 ps
16-bit comparator	XOR/AND/OR chain with early-out	50-300 ps

- Rough delays for TSMC 16nm
- Custom ASICs are much faster than FPGAs
  - Easily 5 to 10x faster

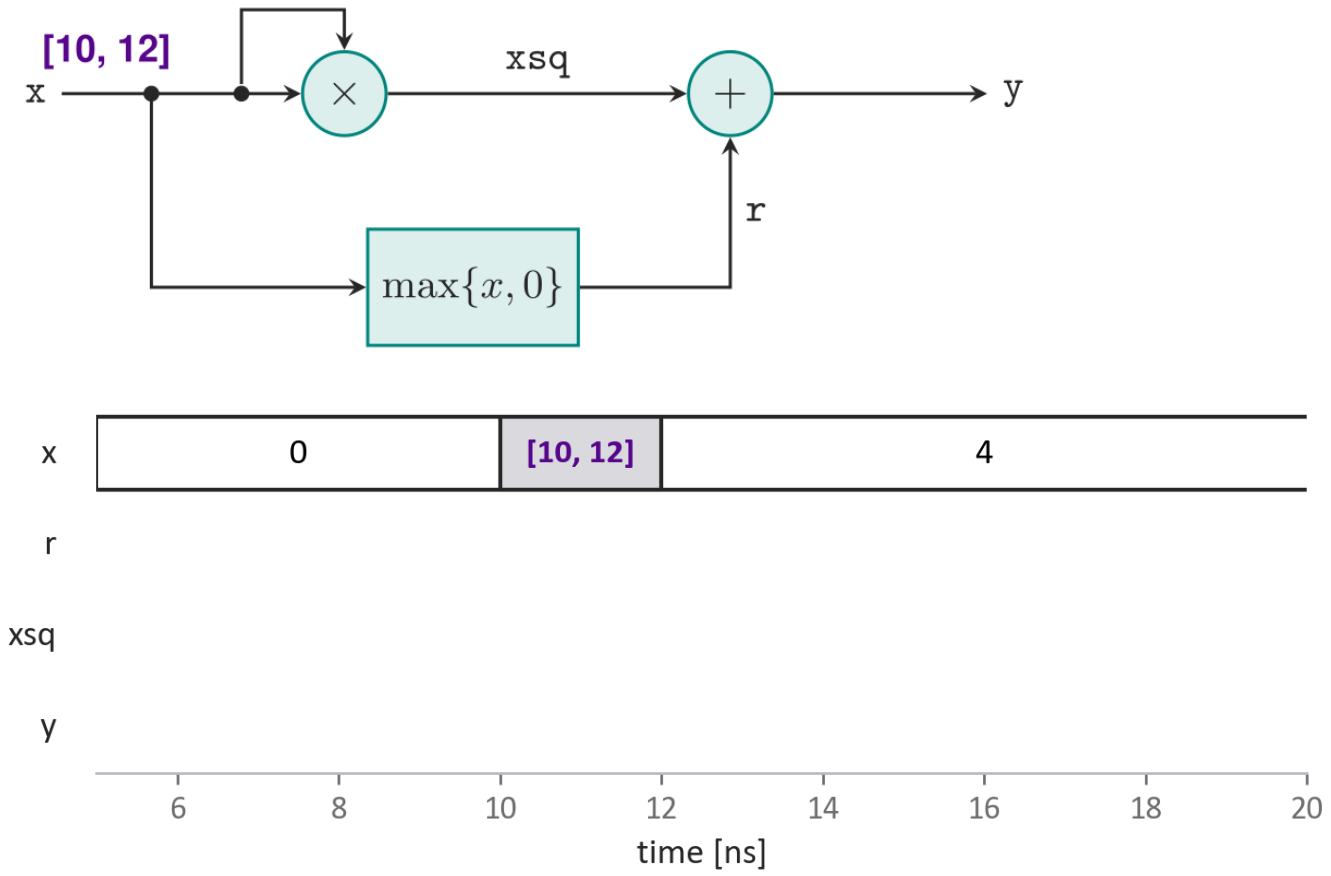
Timing Diagram with Multiple Paths

- Now a circuit with **two paths** into the adder
  - One through xsq = x·x
  - One through the ReLU: r = max(x, 0)
- How does the timing diagram work?



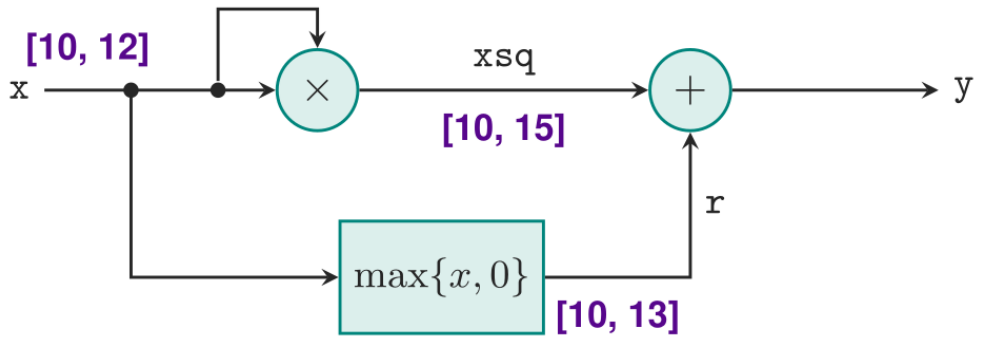

```
xsq = x * x;
r = (x > 0) ? x : 0;
y = xsq + r;
```

Multi-Path Timing: Input



- x changes from 0 to 4
- It is in transition, unknown, for t ∈ [10, 12]

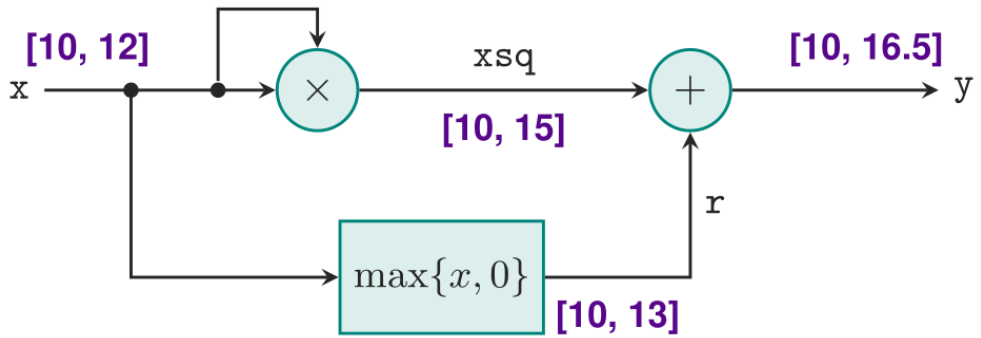
Multi-Path Timing: First Steps

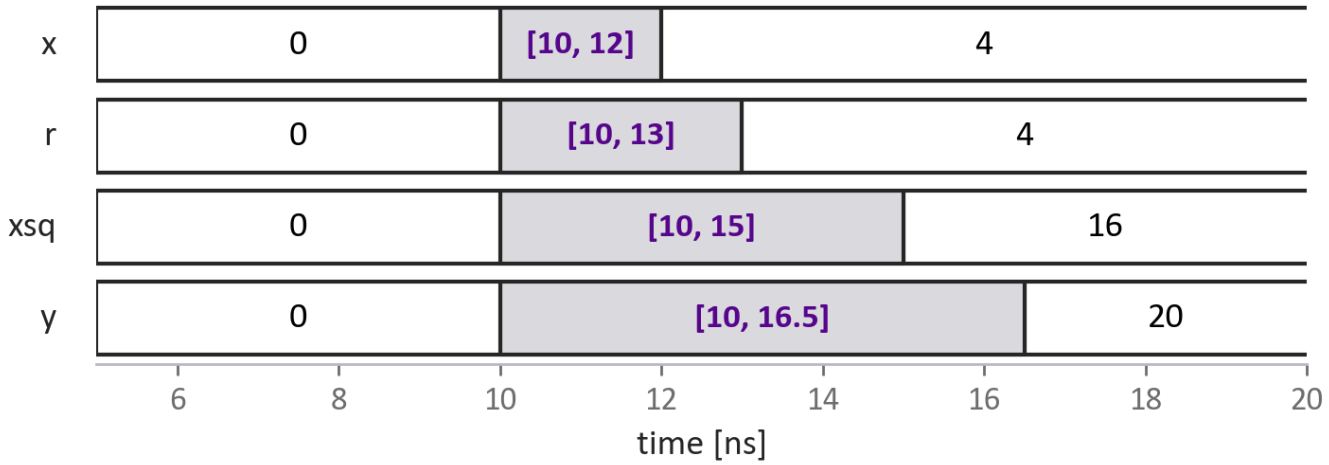




- xsq: the multiplier takes 3 ns
  - Unknown in **[10, 15]**, then 16
- r: the ReLU takes 1 ns
  - Unknown in **[10, 13]**, then 4
- The two paths settle **in parallel**

Multi-Path Timing: Output





- y goes unknown when **either** input does: at 10
- It settles once **both** are stable: xsq at 15, r at 13
- The adder takes 1.5 ns: y is stable at **16.5**, value 20
- **The latest input sets the delay**

Multi-Path Timing

PRACTICE

x is 0 for a long time before 0 ns, unknown from 10 ns, and 4 from 12 ns. a = 3 and b = 6. Delays: multiplication 3 ns, addition 2 ns, max 1 ns.

(a) What are the stable values of z1, z2 and y before 10 ns?

(b) What are their stable values after 12 ns, once every delay has passed?

(c) When does each of z1, z2 and y become unknown, and when is it stable again?

```
z1 = a*x;
z2 = x+b;
y = max(z1, z2);  // the larger
```

Time [ns]	0	10	12
x	0	?	4

Also in the LLM grader: Unit 6 › Multi-path timing

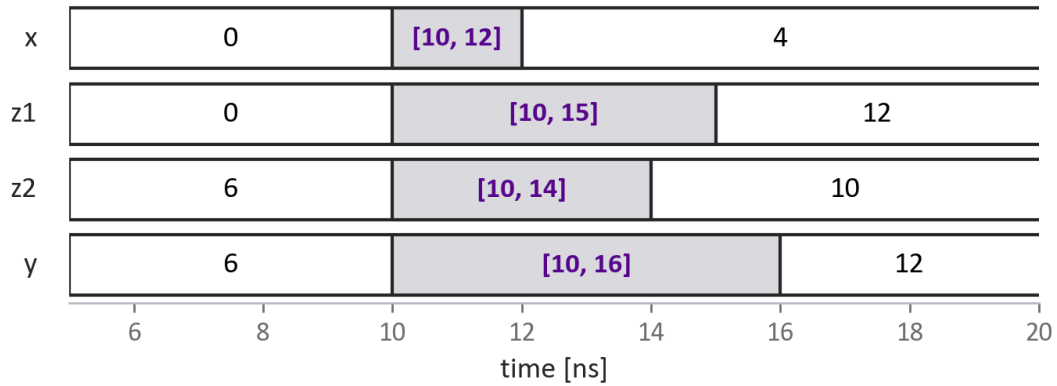
Multi-Path Timing

SOLUTION

(a) z1 = 0, z2 = 6, y = max(0, 6) = 6

(b) **z1 = 12, z2 = 10, y = 12**

(c) All go unknown at **10 ns**. x settles at 12 ns, so:
z1 at **15 ns** (12 + 3), z2 at **14 ns** (12 + 2)
y at **16 ns**: the later of the two, plus 1



Common error

Counting from 10 ns, when x first changes: 13, 12 and 14. The delay starts when x is stable again.

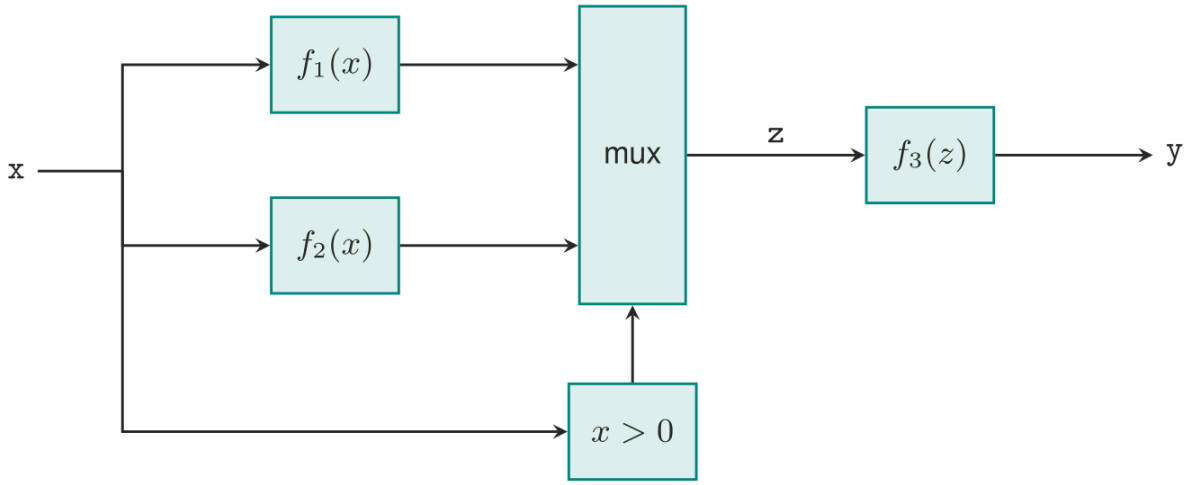
Also in the LLM grader: Unit 6 › Multi-path timing

Conditional Circuits

- Consider a conditional block
- How is it implemented in hardware?
- What is its maximum propagation delay?

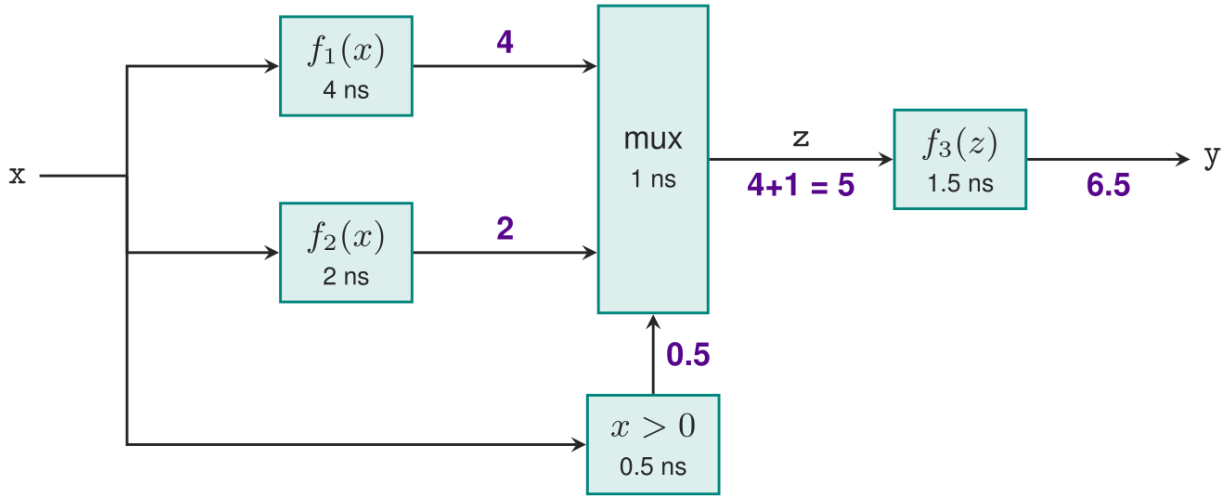
```
// mux delay = 1
if (x > 0) begin          // delay = 0.5
    z = func1(x);         // delay = 4
end else begin
    z = func2(x);         // delay = 2
end
y = func3(z);             // delay = 1.5
```

Conditionals as Multiplexers



- **Multiplexer**: passes one of its inputs, chosen by a condition
- The inputs and the condition are **all computed in parallel**
  - Each needs its own hardware
  - The branch not taken saves nothing
- The output settles after the **slowest** input, plus the mux delay

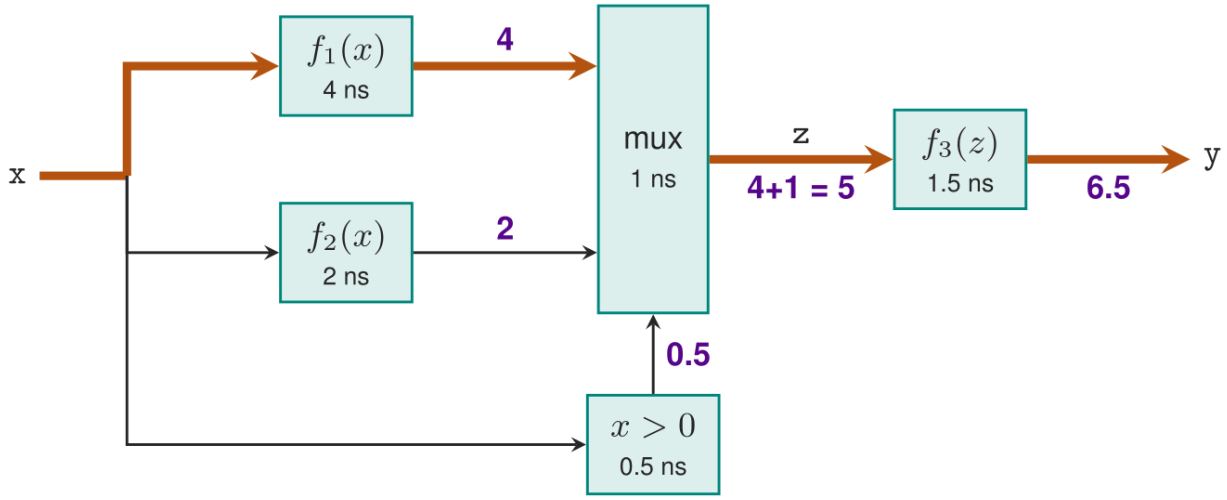
Propagation Delay with a Multiplexer



- Arrival times in **violet**
- The slowest mux input is z = $f_1(x)$: 4 ns
  - Not the branch x happens to select: STA covers every input
- The mux adds 1 ns: z is stable at **5 ns**
- y is stable 1.5 ns later: **6.5 ns**

Homework in the LLM grader: Unit 6 › Timing with multiplexer

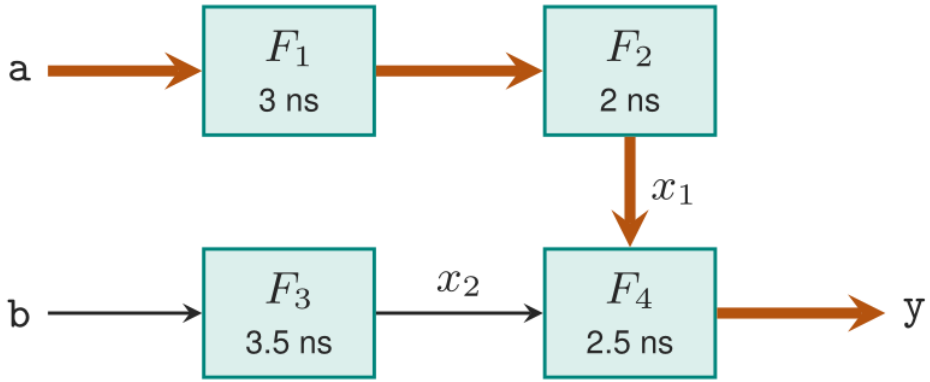
Critical Path



- **Critical path**: the path with the largest propagation delay
  - Here: x → f_1 → mux → f_3 → y
- To speed up the circuit, shorten the critical path
  - A faster f_1 helps
  - A faster f_2 does not

Critical Path Example

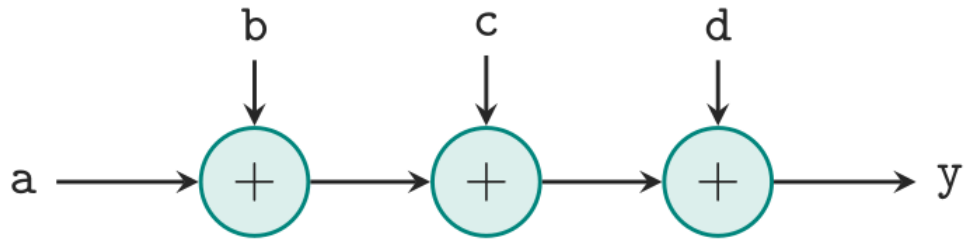
- Functions F_1 … F_4, with $x_1 = F_2(F_1(a))$, $x_2 = F_3(b)$, $y = F_4(x_1, x_2)$
- **Path 1**: $F_1 \rightarrow F_2 \rightarrow F_4$
  - 3 + 2 + 2.5 = 7.5 ns: the longest, so the **critical path**
- **Path 2**: $F_3 \rightarrow F_4$
  - 3.5 + 2.5 = 6 ns: not critical



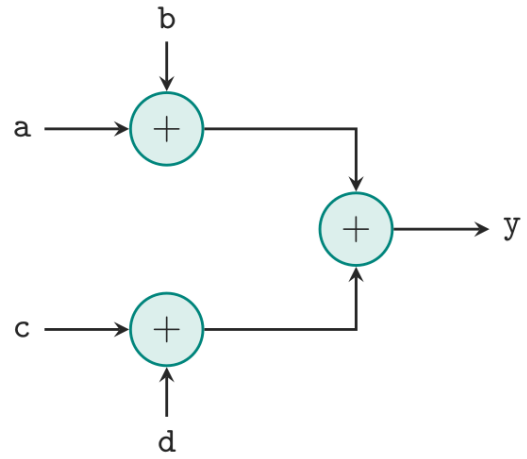
Delay Depends on the Implementation

- Example: y = a + b + c + d, with each addition taking D
- Synthesis tools look for a low-delay implementation

IN SERIES: DELAY 3D



AS A TREE: DELAY 2D



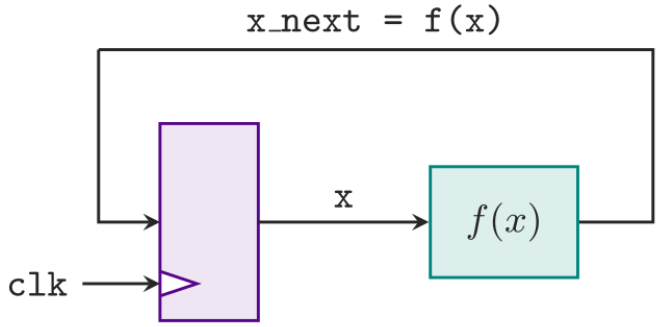
Outline

- Static Timing Analysis in Combinational Logic
- Static Timing Analysis in Sequential Logic
- Sequential Logic Timing Diagrams
- VCD Files and Timing Diagrams

Synchronous Logic

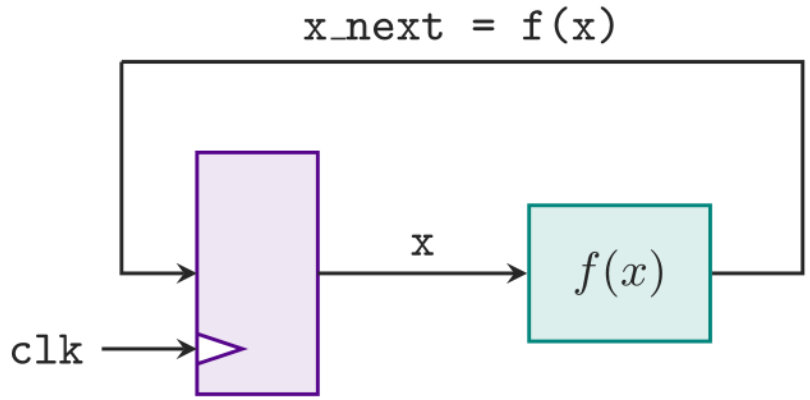
- Now: static timing analysis of synchronous logic
- In this unit we assume
  - All registers are clocked on the same rising edge
  - The clock reaches every register at the same time: no **clock skew**
  - Reset paths are not analyzed

```
always_comb begin
    x_next = f(x);
end
always_ff @(posedge clk) begin
    x <= x_next;
end
```



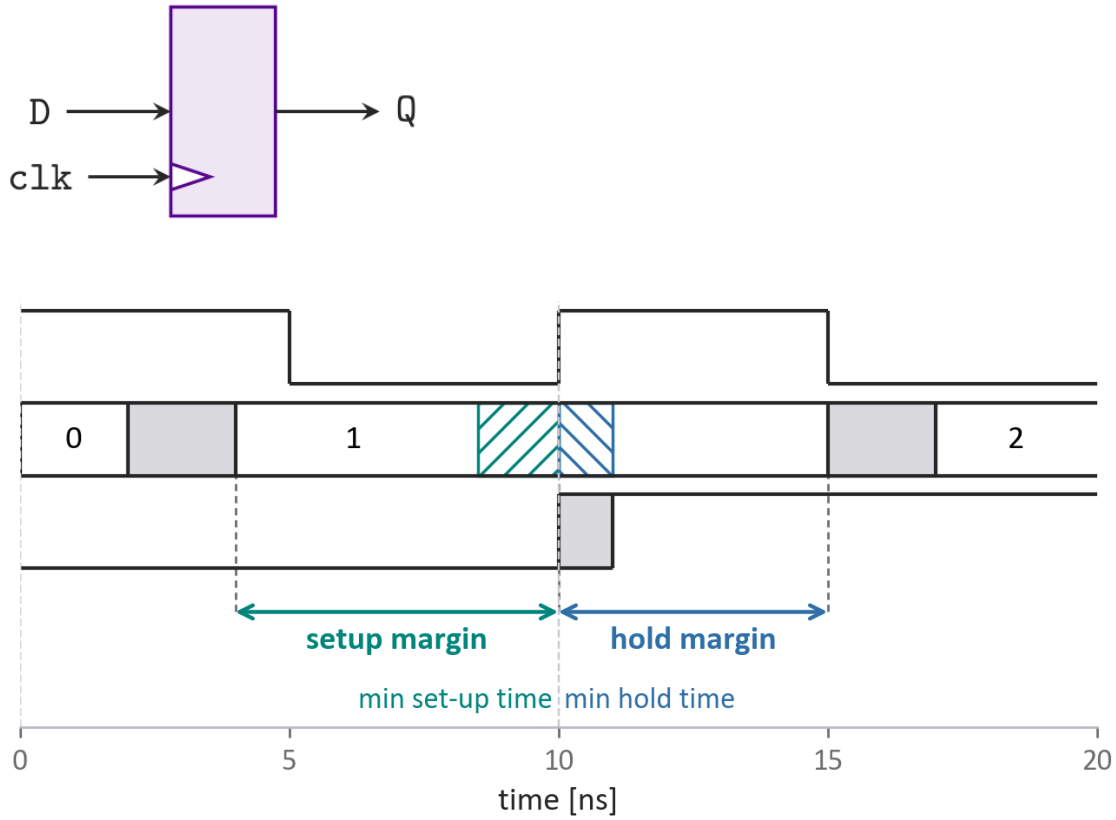
STA for Synchronous Logic

- STA checks every **register-to-register** path: from a register output to a register input
- Two conditions
  - **No path arrives too late**: the next value must be stable a **setup time** before the next rising edge
  - **No path arrives too early**: a register's input must stay stable a **hold time** after the edge



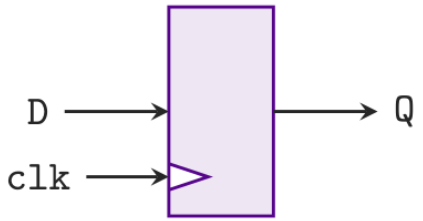
Setup and Hold Time

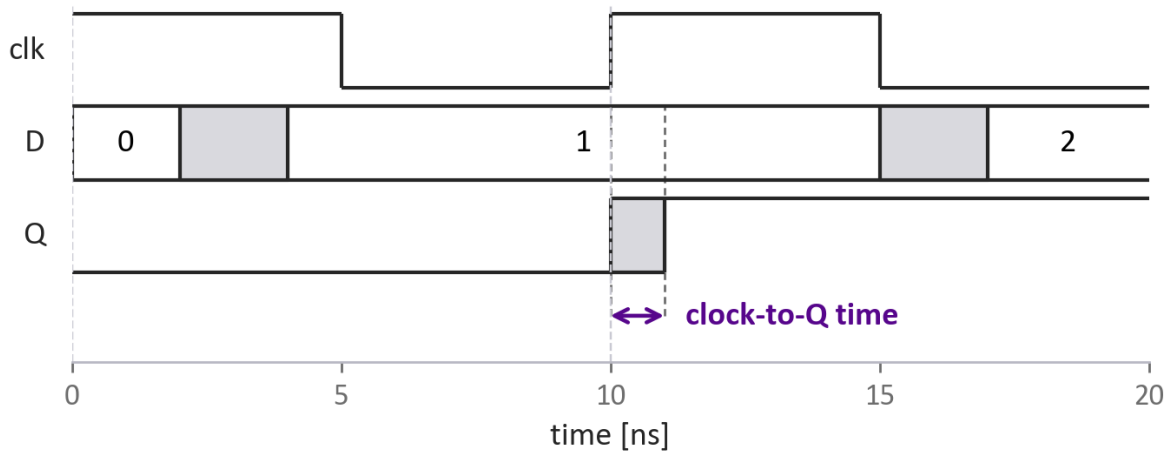
- Registers are usually **D flip-flops**
- Correct operation needs D stable around the rising edge
- **Setup margin**: from D stable to the edge
  - Must exceed the **setup time**
- **Hold margin**: from the edge to D changing
  - Must exceed the **hold time**



Clock-to-Q Time

- **Clock-to-Q time**: from the rising edge to a stable output Q
  - Loosely called the register delay
- The time to pass D through to Q





Typical Times

Technology	Clock-to-Q	Setup	Hold	Typical clock period
Low-end FPGA (PYNQ-Z2 / Zynq-7000, Artix-7)	300–600 ps	150–300 ps	0–150 ps	5–10 ns (100–200 MHz)
High-end FPGA (UltraScale+, Stratix 10)	150–300 ps	100–200 ps	0–120 ps	1.5–3 ns (330–670 MHz)
ASIC, modern node (7–28 nm)	20–80 ps	20–60 ps	−20 to +40 ps	0.3–1 ns (1–3 GHz)

Modern nodes can have a **negative** hold time: the input may change slightly before the edge.

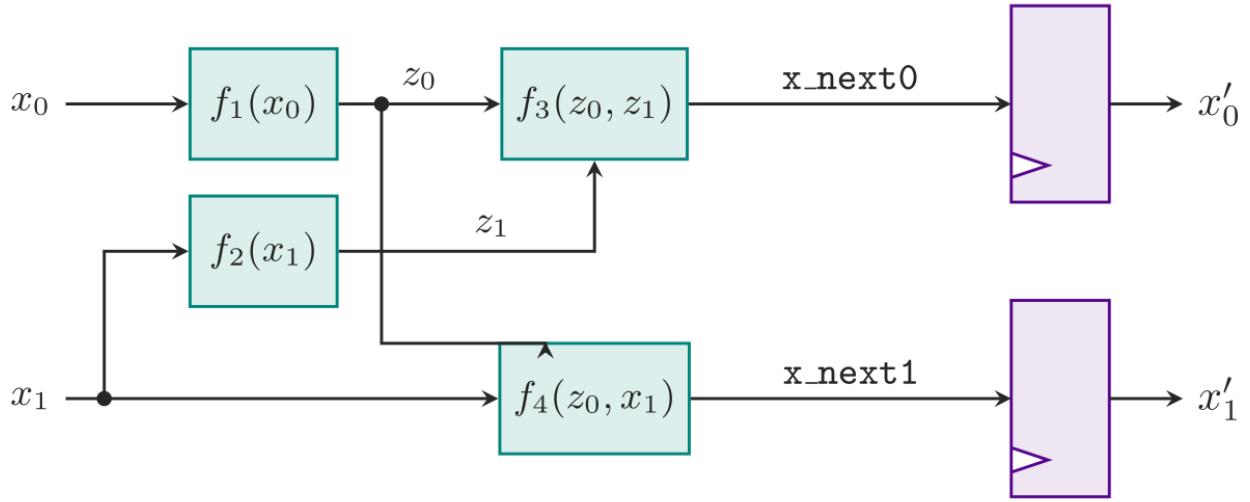
Register-to-Register Paths

```
always_comb begin
    z0 = func1(x0);
    z1 = func2(x1);
    x_next0 = func3(z0, z1);
    x_next1 = func4(z0, x1);
end
always_ff @(posedge clk) begin
    x0 <= x_next0;
    x1 <= x_next1;
end
```

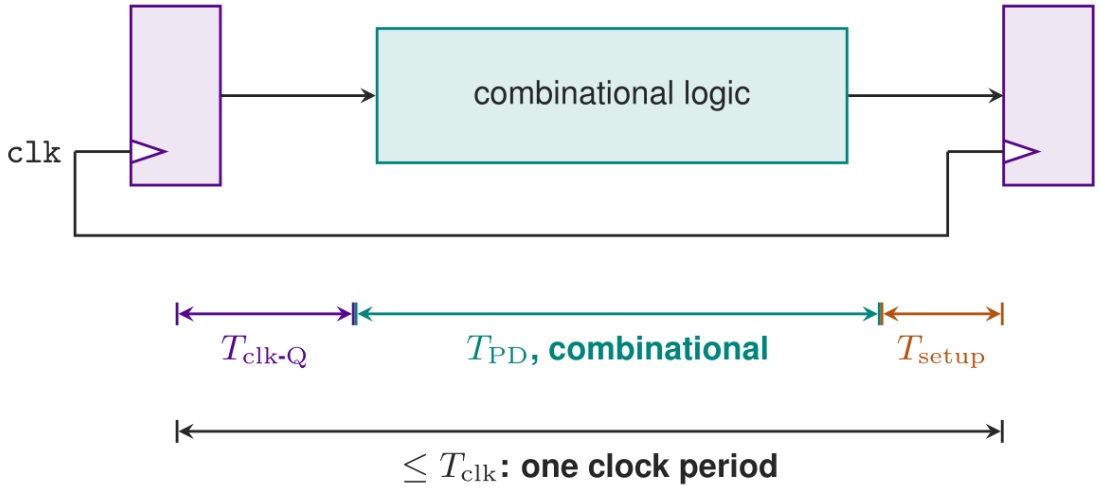
FOUR PATHS

```
x0 → f1 → f3 → x0'
x0 → f1 → f4 → x1'
x1 → f2 → f3 → x0'
x1 → f4 → x1'
```

- **Register-to-register path**: from any register output to any register input
- Primed values x_0', x_1' are the values in the **next** cycle



Verifying the Setup Time



- For now, assume the hold condition is met
- Every path must arrive a setup time before the next edge
- A path's delay has two parts
  - $T_{clk\text{-}Q}$: the register that launches it
  - T_{PD}: the combinational logic
- T_{crit}: the largest T_{PD}, along the critical path

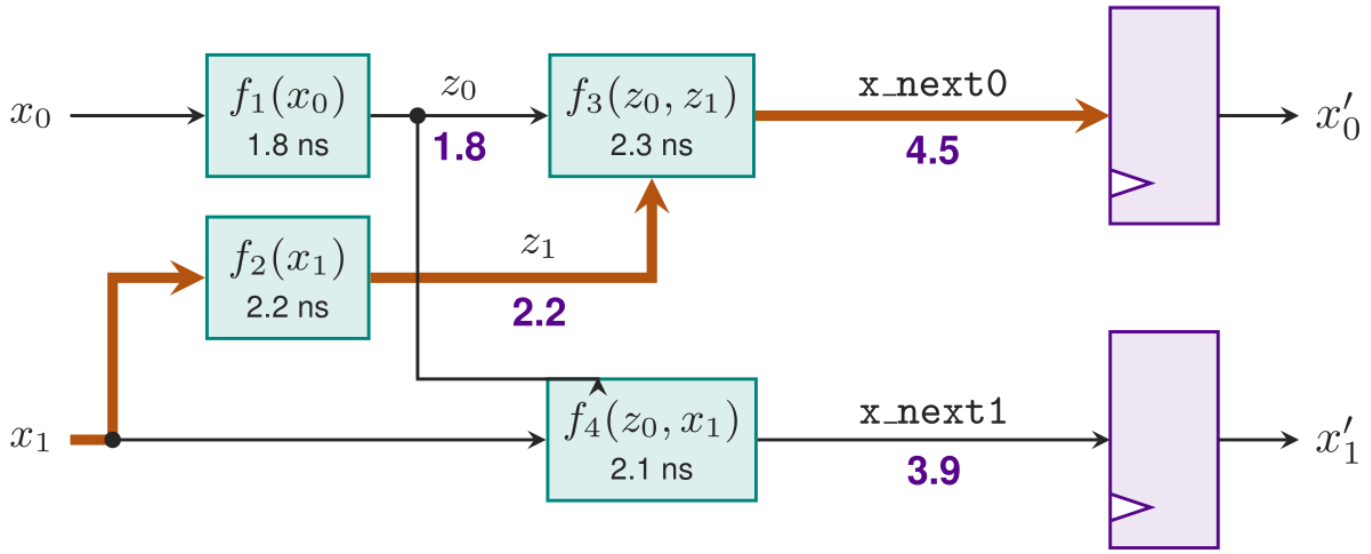
To meet timing: $T_{clk} \geq T_{crit} + T_{clk\text{-}Q} + T_{setup}$

Meeting or Failing Timing

$$T_{clk} \geq T_{crit} + T_{clk\text{-}Q} + T_{setup}$$

- T_{clk}: the clock period; $f_{clk} = 1 / T_{clk}$, the clock frequency
- T_{crit}: the delay of the critical path through the combinational logic
- T_{setup}: the minimum setup time; $T_{clk\text{-}Q}$: the clock-to-Q time
- If it holds, the design **meets timing**
- Otherwise it **fails timing**: a **timing violation**

Example: Finding the Critical Path



Component	Value [ns]
f_1 PD	1.8
f_2 PD	2.2
f_3 PD	2.3
f_4 PD	2.1
$T_{clk\text{-}Q}$	0.2
T_{setup}	0.1

- Arrival times computed block by block, as for combinational logic
- x_next0: max(1.8, 2.2) + 2.3 = 4.5; x_next1: 1.8 + 2.1 = 3.9
- Critical path $x_1 \rightarrow f_2 \rightarrow f_3 \rightarrow x_0'$: **$T_{crit}$ = 4.5 ns**

Example: Maximum Clock Frequency

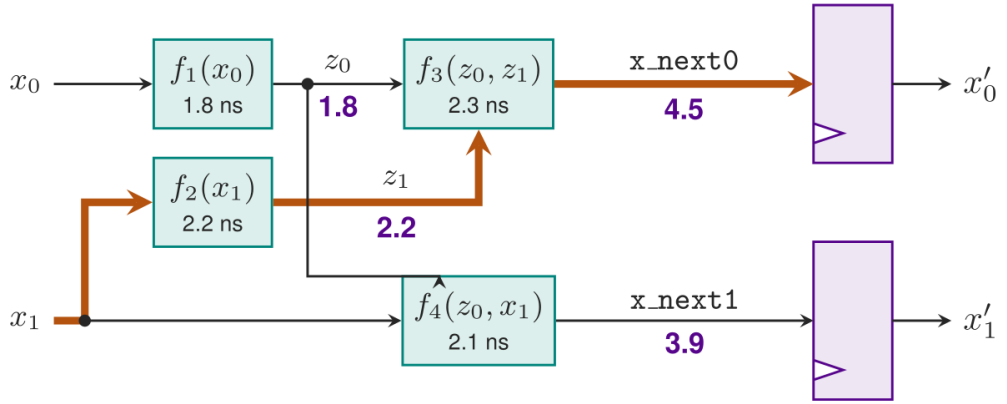
From the previous slide, T_{crit} = 4.5 ns

So to meet timing

$$T_{clk} \geq T_{crit} + T_{clk\text{-}Q} + T_{setup}$$
$$= 4.5 + 0.2 + 0.1 = \mathbf{4.8\ ns}$$

Maximum clock frequency

$$f_{clk} = 1 / T_{clk} \leq 1 / 4.8\ \text{ns} \approx \mathbf{208\ MHz}$$

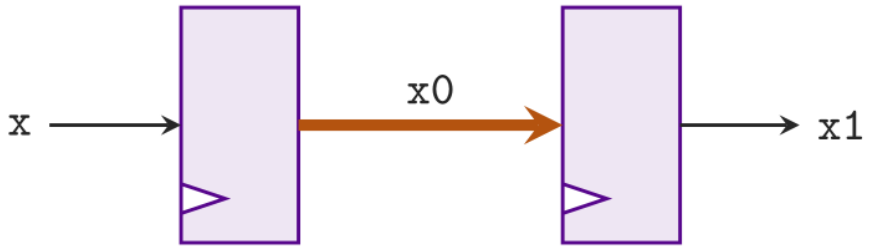


Homework in the LLM grader: Unit 6 › Critical path

Meeting the Minimum Hold Time

- Consider a simple **delay line**
- A naive implementation has a very short path from x0 to the input of x1
  - It could violate the minimum hold time
- Place and route fixes this: it inserts delay where a path is too short
- So in a reasonable synchronous design, the RTL need not worry about hold time

```
always_ff @(posedge clk) begin
    x0 <= x;
    x1 <= x0;
end
```

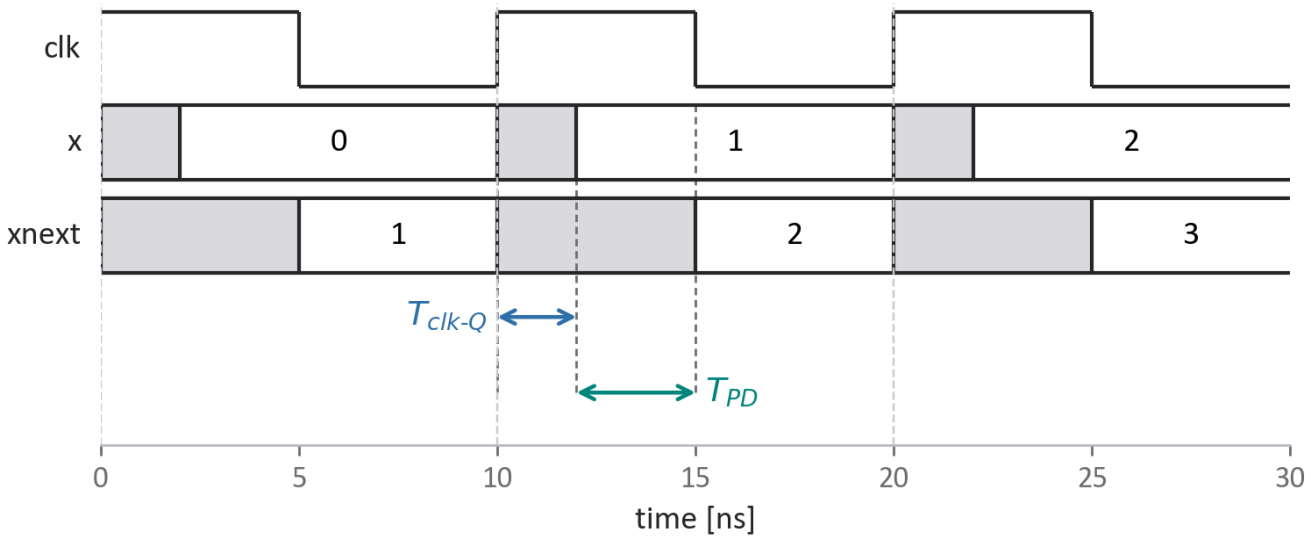


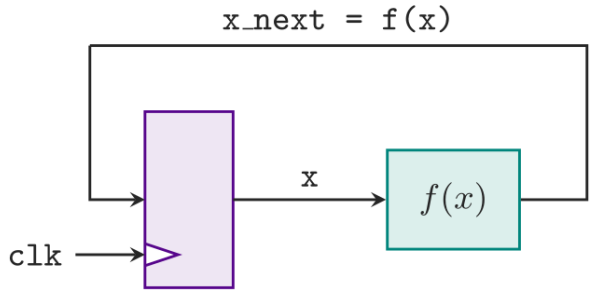
short path: nothing between the registers to delay it

Outline

- Static Timing Analysis in Combinational Logic
- Static Timing Analysis in Sequential Logic
- Sequential Logic Timing Diagrams
- VCD Files and Timing Diagrams

Synchronous Logic in Detail

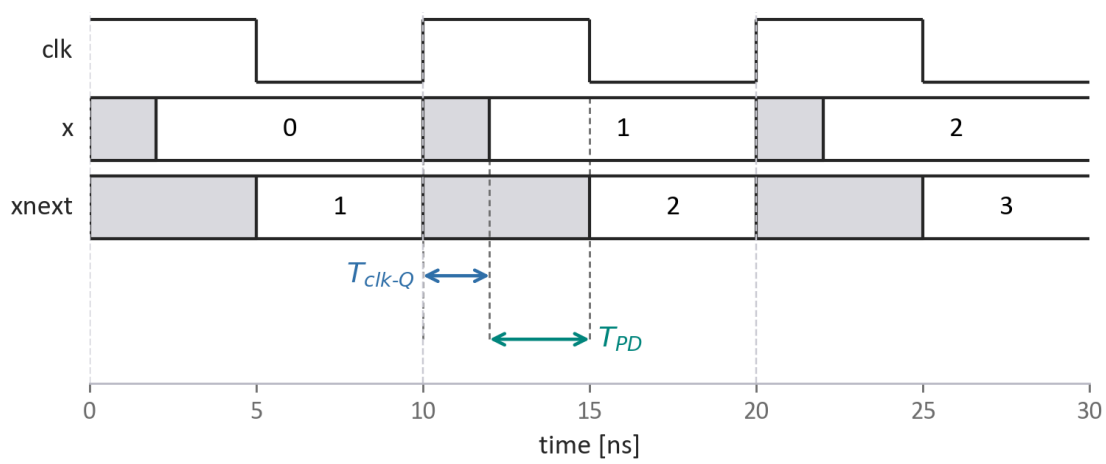




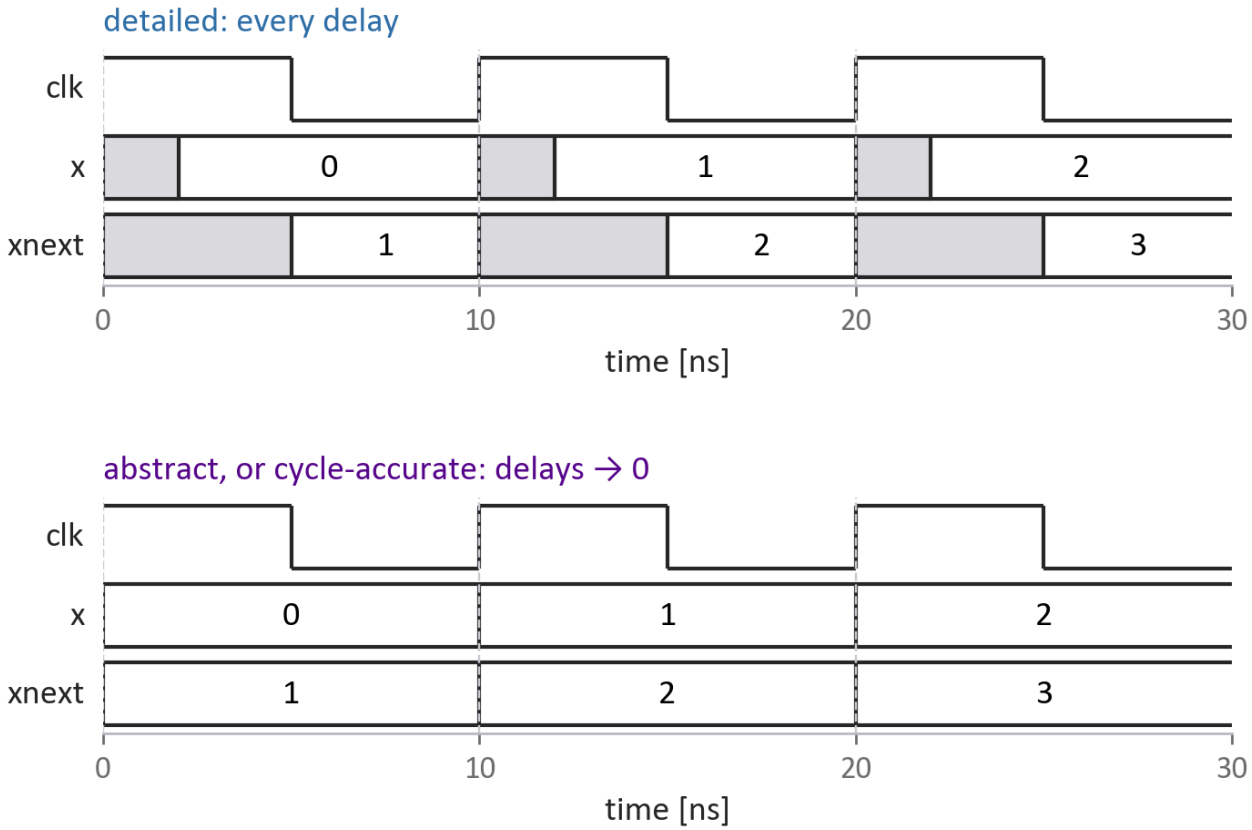
- A counter: f(x) = x + 1
- Every synchronous circuit has two delays
  - **Clock-to-Q**: edge → register output; 2 ns here
  - **Combinational PD**: register output → next value; 3 ns here

Detailed Timing Diagram

- A diagram with every delay is a **detailed** timing diagram
  - Sometimes called the real timing diagram
- It shows register delays and every propagation delay
- Good for reasoning about precise timing: clock speed, throughput, …
- But too much detail to see what the circuit **does**



Cycle-Accurate Timing Diagram



- **Abstract** or **cycle-accurate** timing diagram
  - Every delay → 0
  - No unknown (X) regions
- Simpler to see what the circuit does
- Each cycle shows the value just before the rising edge that ends it

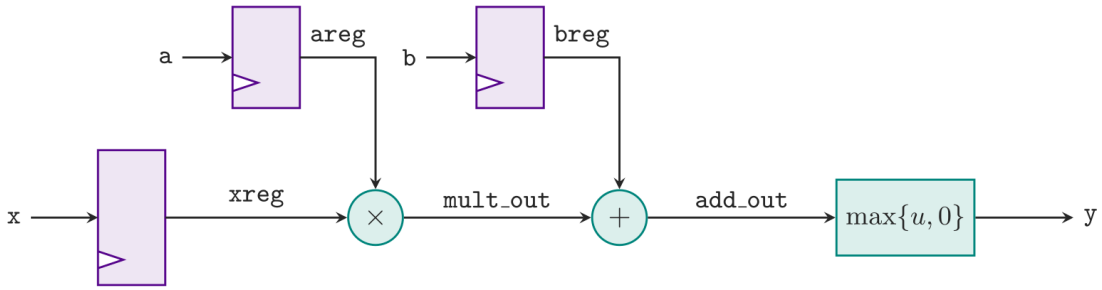
ReLU + Linear: Synchronous Module

- Now the synchronous version of ReLU + linear, as in Unit 2
- Two ports are added
  - **clk**: the clock
  - **rst_n**: an active-low reset

```
module relu_lin (
    input  logic clk,
    input  logic rst_n,
    input  int   x,
    input  int   a,
    input  int   b,
    output int   y
);
```

Synchronous Implementation

- The same implementation as in Unit 2
- Next values computed in an always_comb block

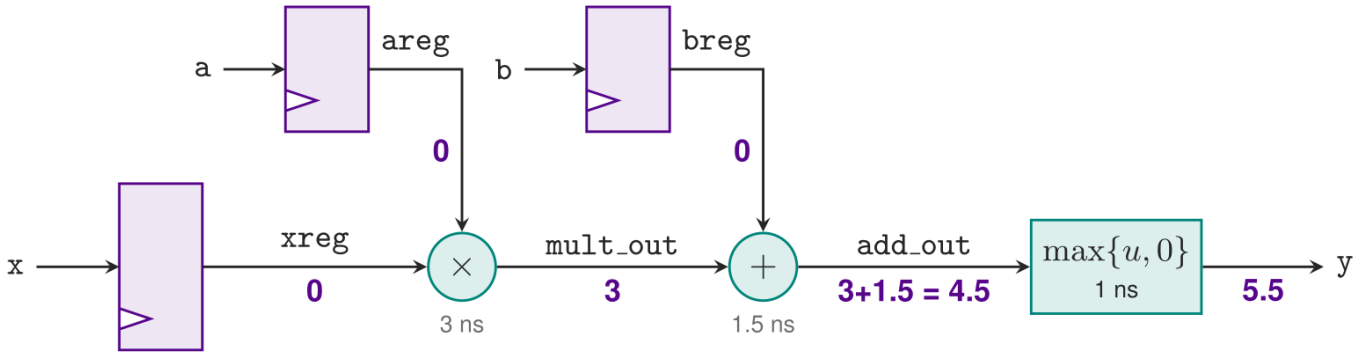



```
int x_reg, a_reg, b_reg;
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        x_reg <= 0;  a_reg <= 0;  b_reg <= 0;
    end else begin
        x_reg <= x;  a_reg <= a;  b_reg <= b;
    end
end
always_comb begin
    int mult_out, add_out;
    mult_out = x_reg * a_reg;
    add_out = mult_out + b_reg;
    y = (add_out > 0) ? add_out : 0;
end
endmodule
```

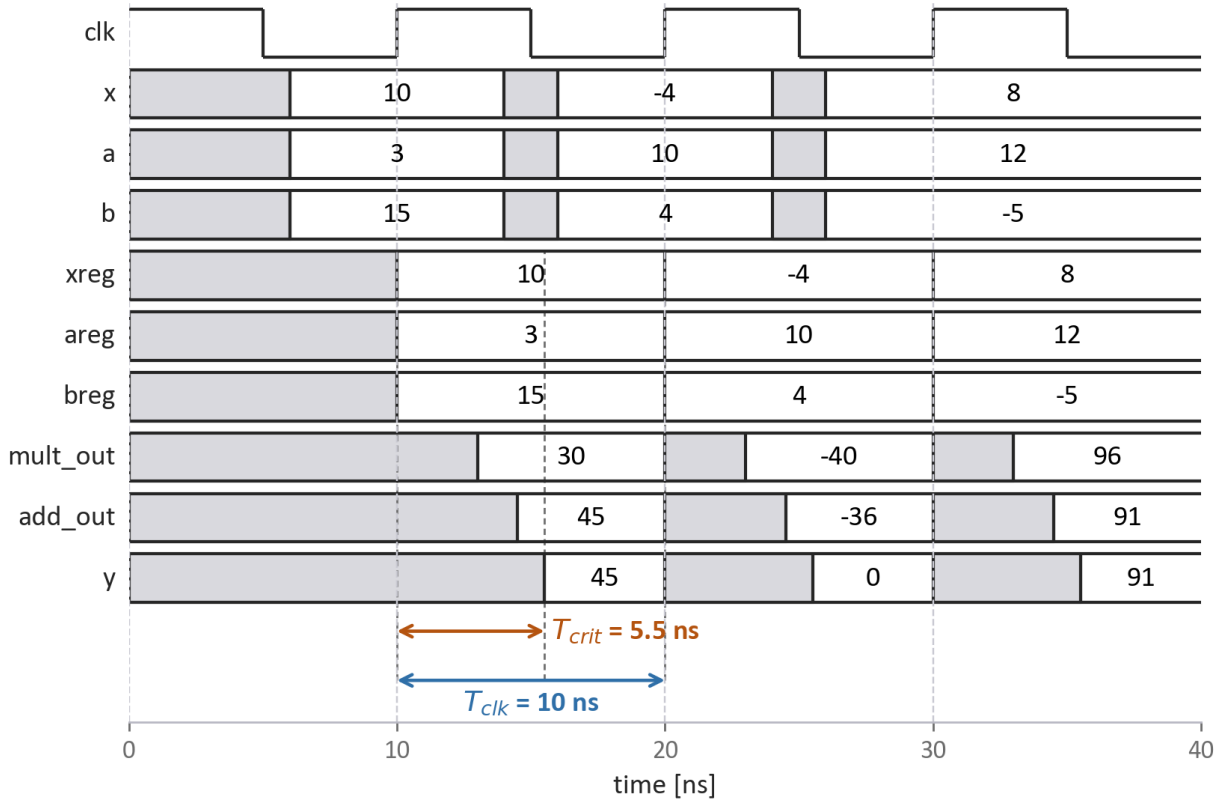
STA of the Synchronous Module

- Assume the delays in the table
- Ignore the clock-to-Q time for now
- The arrival time at each stage is in **violet**

Hardware block	Max prop delay
Register	0 ns
Multiplier	T_{mult} = 3 ns
Adder	T_{add} = 1.5 ns
Max operator	T_{max} = 1 ns
Total: critical path	**T_{crit} = 5.5 ns**



ReLU + Linear Timing Diagram



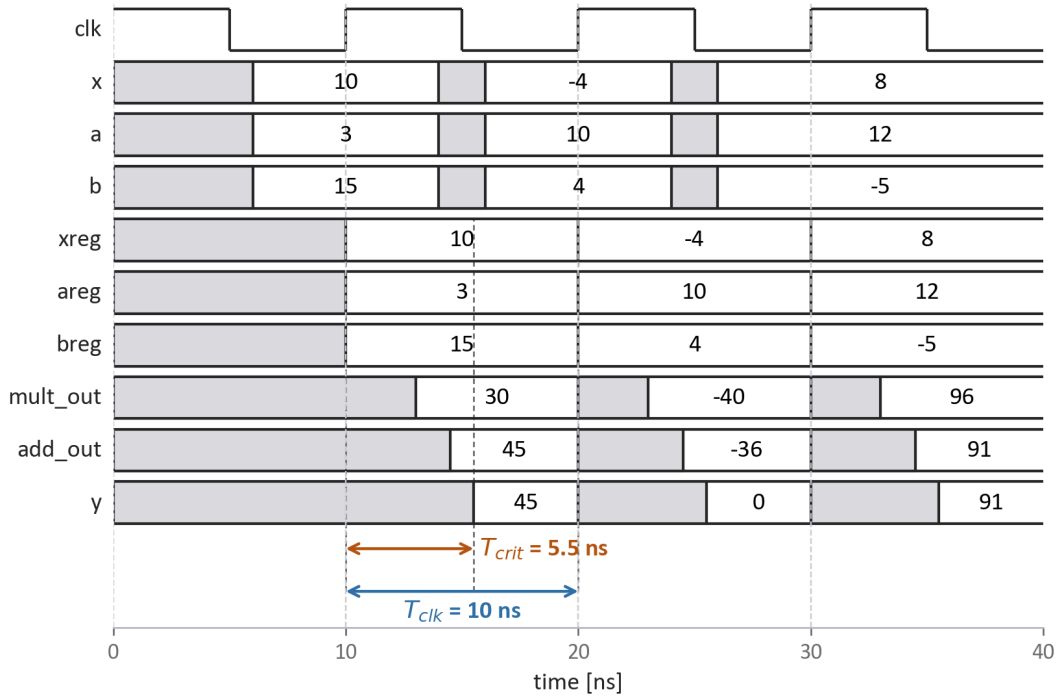
Hardware block	Max prop delay
Register	0 ns
Multiplier	T_{mult} = 3 ns
Adder	T_{add} = 1.5 ns
Max operator	T_{max} = 1 ns
Total: critical path	**T_{crit} = 5.5 ns**

Meets timing: T_{clk} = 10 ns > T_{crit} = 5.5 ns

y settles 5.5 ns after each rising edge, well before the next one.

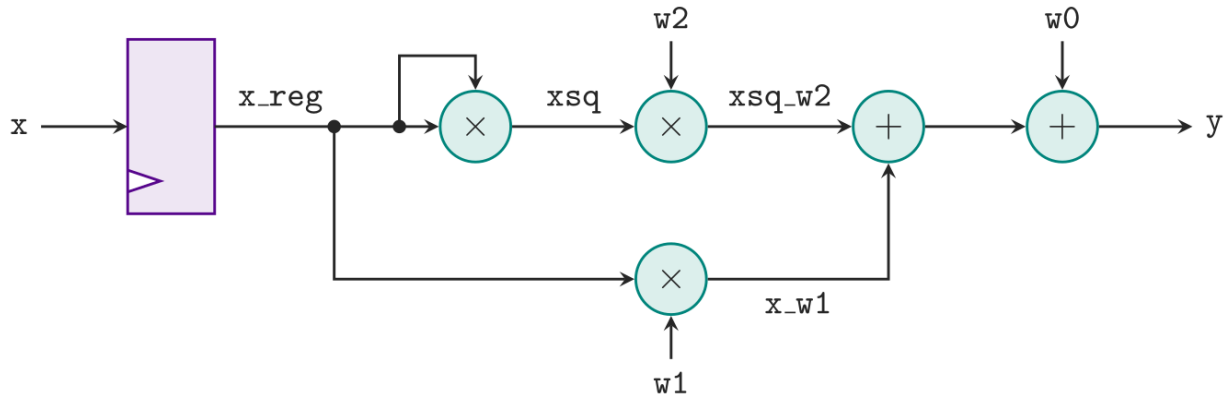
Maximum Clock Frequency Example

- With clock-to-Q and setup time ignored, the minimum clock period is
  - $T_{clk} = T_{crit} = 5.5$ ns
- Maximum clock frequency
  - $f_{clk} = 1 / 5.5$ ns $\approx$ **182 MHz**
- With $T_{clk\text{-}Q}$ and T_{setup}, it would be lower



Quadratic Example

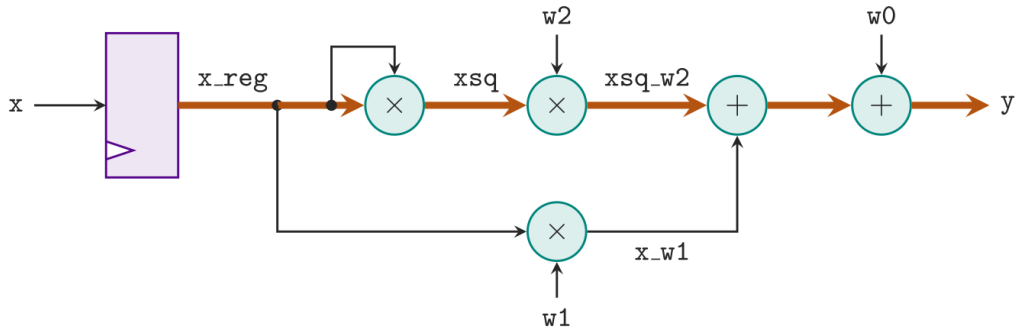
- A quadratic: $y = w_2x^2 + w_1x + w_0$
- w_0, w_1, w_2 are fixed parameters
- A single-cycle implementation, right




```
module quad_func #(
    parameter int w2 = 0,
    parameter int w1 = 0,
    parameter int w0 = 0
) (
    input  logic clk,
    input  int   x,
    output int   y
);
    int x_reg;
    always_ff @(posedge clk) begin
        x_reg <= x;
    end
    always_comb begin
        int xsq, xsq_w2, x_w1;
        xsq    = x_reg * x_reg;
        xsq_w2 = w2 * xsq;
        x_w1   = w1 * x_reg;
        y      = xsq_w2 + x_w1 + w0;
    end
endmodule
```

Finding the Critical Path

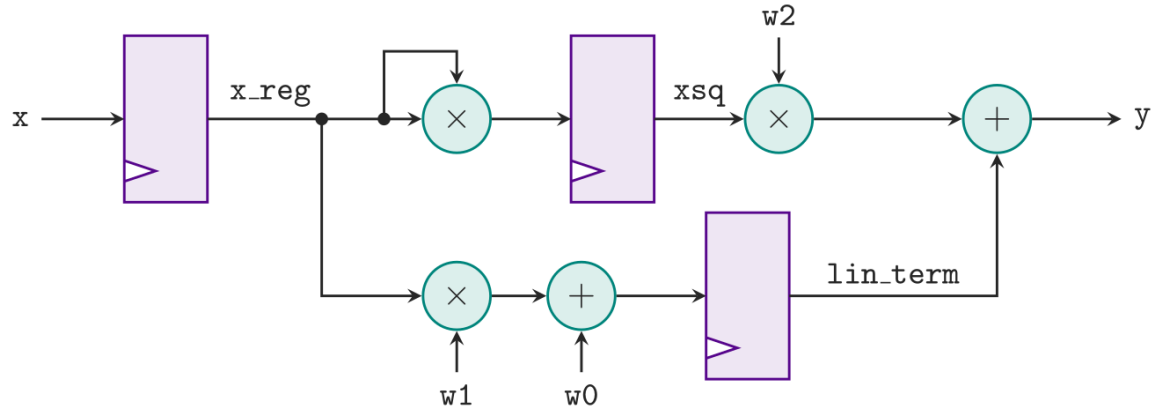
Hardware block	Example value
Register	T_{reg} = 1 ns
Multiplier	T_{mult} = 4 ns
Adder	T_{add} = 1.5 ns
Path 1	**1 + 4 + 4 + 1.5 + 1.5 = 12**
Path 2	1 + 4 + 1.5 + 1.5 = 8



- Two paths from x_reg to y
  - Path 1, through xsq and xsq_w2: a register, 2 multiplies, 2 adds: 12 ns
  - Path 2, through x_w1: a register, 1 multiply, 2 adds: 8 ns
- Path 1 is critical
- Maximum clock frequency = 1 / 12 ns ≈ **83 MHz**
- So it **fails timing** at 100 MHz

A Two-Cycle Design

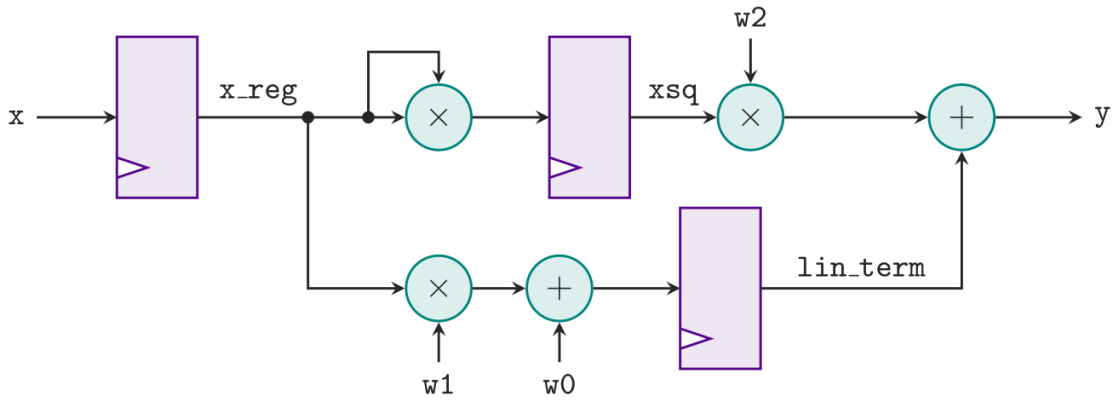
- Improve the clock rate by spreading the computation over **two cycles**
- Register two intermediate terms
  - `xsq = xreg * xreg`
  - `lin_term = w1*xreg + w0`




```
int xreg, xsq, lin_term;
always_ff @(posedge clk) begin
    xreg     <= x;
    xsq      <= xreg * xreg;
    lin_term <= w1 * xreg + w0;
end
always_comb begin
    y = w2 * xsq + lin_term;
end
```

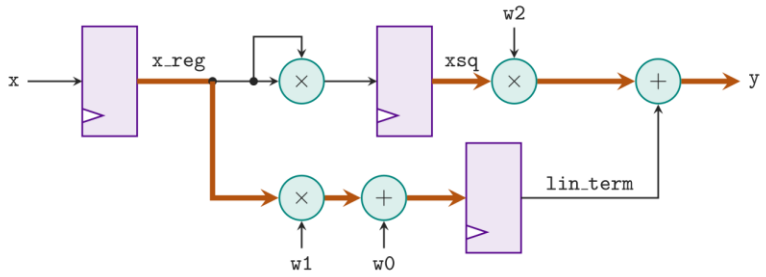
Three Stages

- The design now runs in three stages, over two cycles
- **Stage 0**: register x into xreg
- **Stage 1**: compute and register xsq and lin_term
  - xsq = xreg · xreg
  - lin_term = $w_1 \cdot$xreg + w_0
- **Stage 2**: compute y = $w_2 \cdot$xsq + lin_term



Critical Path with Three Stages

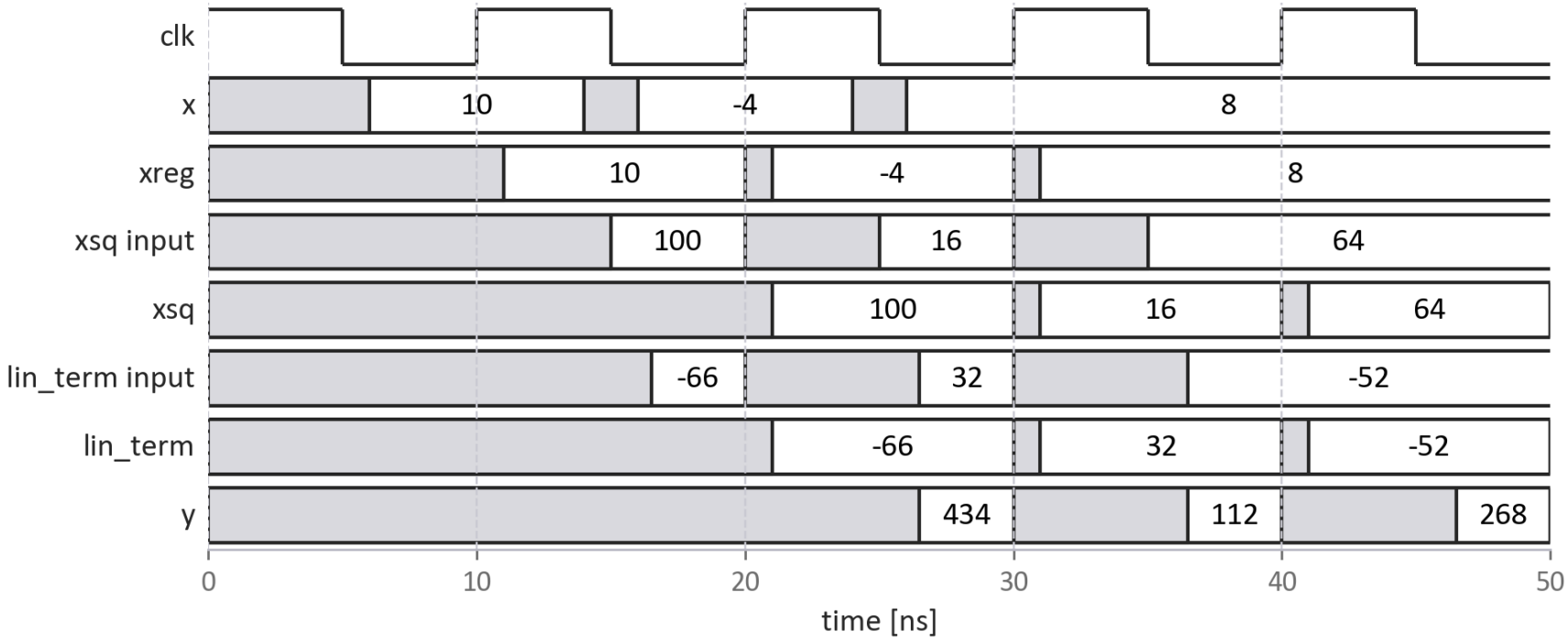
Hardware block	Example value
Register	T_{reg} = 1 ns
Multiplier	T_{mult} = 4 ns
Adder	T_{add} = 1.5 ns
Path 1	1 + 4 = 5
Path 2	**1 + 4 + 1.5 = 6.5**
Path 3	**1 + 4 + 1.5 = 6.5**



Homework in the LLM grader: Unit 6 › Two cycle datapath

- Measure every path from register to register
  - Path 1: x_reg → the xsq register: 1 register, 1 multiply
  - Path 2: x_reg → the lin_term register: 1 register, 1 multiply, 1 add
  - Path 3: xsq, lin_term → y: 1 register, 1 multiply, 1 add
- Critical path delay: 6.5 ns, down from 12
- Maximum clock frequency = 1 / 6.5 ns ≈ **154 MHz**

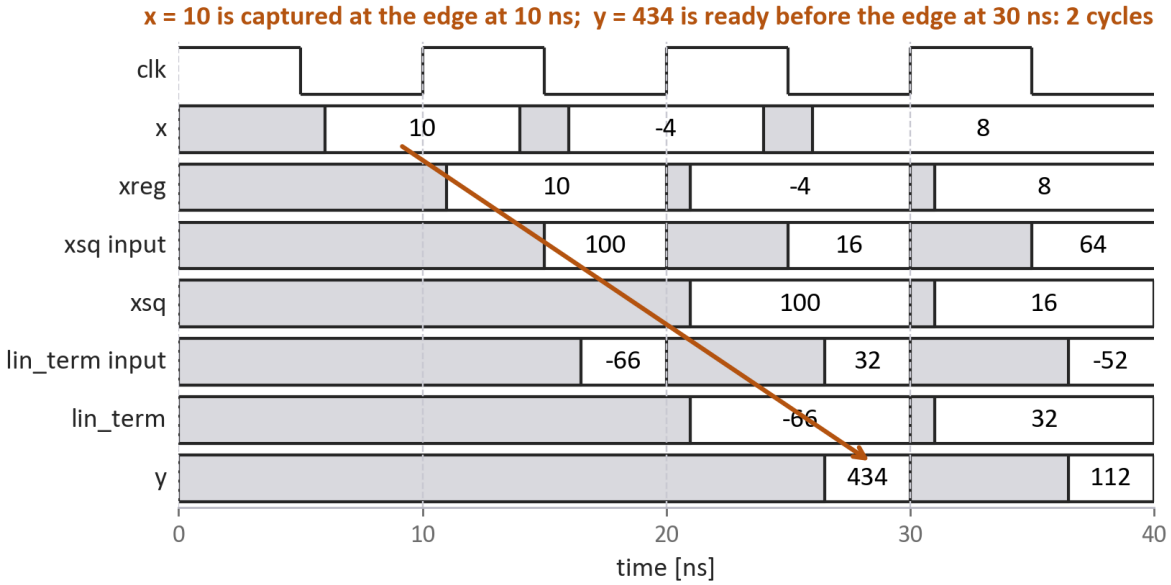
Two-Cycle Timing Diagram



- $w_0, w_1, w_2 = 4, -7, 5$
- x = 10, −4, 8
- y = 434, 112, 268

Latency, Throughput and Pipelining

- Run the design at a 100 MHz clock
- **Latency**
  - An input captured at one rising edge is out before the edge two cycles later
  - Latency = 2 cycles = 20 ns
- **Throughput**
  - A new input every cycle: 100 M inputs/s
- This is a **pipeline**: several inputs are in progress at once



Performance Comparison

	Single-cycle design	Two-cycle design
Critical path	12 ns	6.5 ns
Maximum clock frequency	≈ 83 MHz	≈ 154 MHz
Maximum throughput	≈ 83 M inputs/s	≈ 154 M inputs/s
Latency	1 cycle = 12 ns	2 cycles = 13 ns

Pipelining: for a small increase in latency, a large gain in throughput.

The Demo: Latency and the Clock

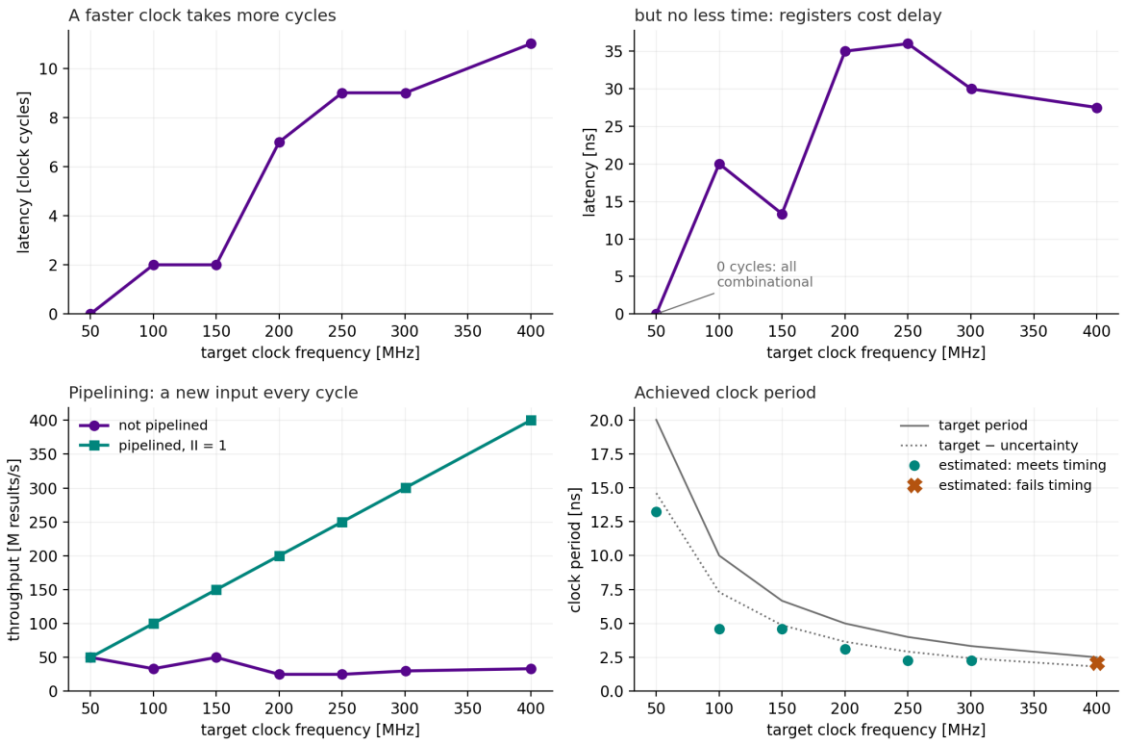
sdrangan.github.io/hwdesign — Demos › Timing and the Clock

RUN THE SWEEP

```
cd demos/timing/quad
python quad_build.py --list-steps
python quad_build.py --force-step sweep
```

WHAT TO WATCH FOR

- The quadratic with **no AXI**: plain wires and `ap_start` / `ap_done`
- Latency: **0 cycles** at 50 MHz, 2 at 100 MHz, 9 at 300 MHz
- In ns it does **not** fall: 20 ns at 100 MHz, 30 ns at 300 MHz
- Fails timing at 400 MHz; pipelined, a new input **every cycle**



Outline

- Static Timing Analysis in Combinational Logic
- Static Timing Analysis in Sequential Logic
- Sequential Logic Timing Diagrams
- VCD Files and Timing Diagrams

Value-Change Points

- Simulation tools trace values over time for large numbers of signals
- Data on each signal is typically stored via value-change points

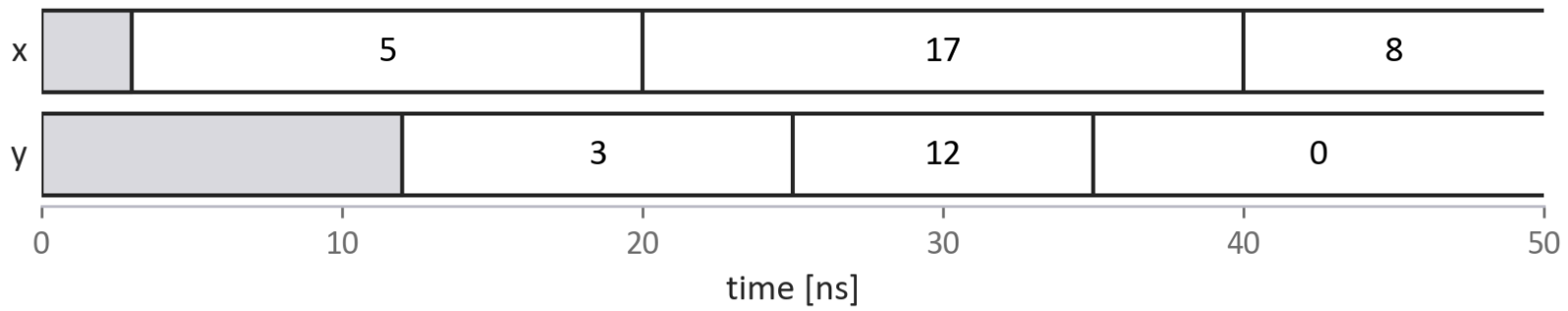
Definition: A signal $x(t)$ has value-change points $\{(t_i, x_i), i = 0, \dots, n-1\}$ if:

$$x(t) = \begin{cases} X & t < t_0 \\ x_i & t_i \le t < t_{i+1}, \quad i = 0, \dots, n-2 \\ x_{n-1} & t \ge t_{n-1} \end{cases}$$

  - Value change points also called value-change events, transitions, …

Drawing Value-Change Points

- A timing diagram can be drawn from the value-change points
- Example with two signals, times in ns
  - x: (3, 5), (20, 17), (40, 8)
  - y: (12, 3), (25, 12), (35, 0)
- Before its first point, a signal is unknown



VCD Files from a Vitis Simulation

- Co-simulation records waveforms in a **.wdb** file
  - An AMD format that only the Vivado viewer opens
- A **VCD** (value change dump) is an open text format
  - A header naming the signals, then the time of every change
- The demo builds re-run the RTL simulation with VCD logging on: the `extract_vcd` step
  - Port signals by default; `--trace-level all` for every internal one

THE EXAMPLE ON THE LAST SLIDE, AS A VCD

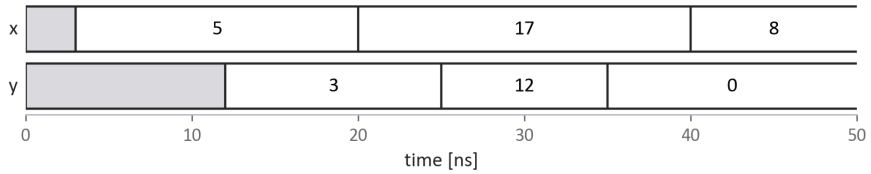
```
$timescale 1ns $end
$scope module top $end
$var wire 32 ! x [31:0] $end
$var wire 32 " y [31:0] $end
$upscope $end
$enddefinitions $end
#3
b101 !
#12
b11 "
#20
b10001 !
#25
b1100 "
...
```

Drawing Timing Diagrams in Python

- **waveflow** (in pysilicon) reads VCD files and draws timing diagrams; the value "x" is drawn as the grey unknown region

```
from waveflow.utils.timing import SigTimingInfo, TimingDiagram

xsig = SigTimingInfo("x", times=[0, 3, 20, 40],
                     values=["x", "5", "17", "8"])
ysig = SigTimingInfo("y", times=[0, 12, 25, 35],
                     values=["x", "3", "12", "0"])
td = TimingDiagram()
td.add_signals([xsig, ysig])
ax = td.plot_signals(trange=[0, 50], add_clk_grid=False)
ax.set_xlabel("time [ns]")
```



The figure the code draws: the example from two slides back.

The Demo: Timing Diagrams from Vitis

sdrangan.github.io/hwdesign — Demos › AXI4-Streaming › Viewing the Timing Diagram

RUN THE WHOLE FLOW

```
cd demos/stream/avgfilt
python avgfilt_build.py --list-steps
python avgfilt_build.py --through extract_vcd
python avgfilt_build.py --through timing_diagram
```

WHAT TO WATCH FOR

- `extract_vcd` re-runs the RTL simulation with VCD logging switched on
- `timing_diagram` reads the VCD and decodes the stream off the wires
- A sample moves on a rising edge where TVALID and TREADY are both high
- The scalar-function demo does the same for an AXI4-Lite IP

Measured on the RTL: a latency of 19 cycles, then 200 samples out in 200 cycles.

The latency and throughput this unit computes by hand, read off a real waveform.